
Master thesis: Anomaly detection methods for the prediction of contracts termination

Degree: Master of Artificial Intelligence and Data Science (IASD)

PSL University - Paris Dauphine

16/09/2025

Student:

Neveu Pierre

Tutors:

Zylinski Steven

Fluteaux Laetitia

Professor:

Airiau Stephane

Contents

1	Introduction	2
1.1	Context and motivation	2
1.2	Problem statement	3
2	State of the art	4
2.1	Generalized Linear Models	4
2.1.1	Definitions and principles	4
2.1.2	Core components	4
2.1.3	Classification and Regression	5
2.2	Anomaly Detecion	7
2.2.1	Definitions and principles	7
2.2.2	Classic models	10
2.2.3	Deep Anomaly Detection	16
3	Experimental results	25
3.1	Presentation of the task	25
3.1.1	Context and objectives	25
3.1.2	Data	26
3.1.3	Experiments settings and modeling choices	31
3.2	Experiments	33
3.2.1	Benchmark of deep anomaly detection models	33
3.2.2	Anomaly detection-based models vs basic classification models	34
3.2.3	The impact of variation in class proportions	37
4	Conclusion	40
4.1	Carried out work	40
4.2	Limitations:	40
4.3	Possible future works:	40
5	Appendix	43
5.1	Requirements	43
5.2	Anomaly Detection existing code	43
5.3	Implementations	43
5.3.1	Models	43
5.3.2	Utils	45

Abstract

Insurers' success hinges not only increasing the number of customers but also on minimizing client churn, which requires leveraging client knowledge. These purposes are usually served by the use of Generalised Linear Models (GLMs), as well as for other machine learning-based tasks. Some have used anomaly detection methods with classical machine learning models to address this type of problems [6]. In recent years, deep learning has been shown to improve state-of-the-art results in many domains, including anomaly detection [15], [11], [14]. The analysis of client churn can be divided in 1) the detection of the termination and 2) the interpretation and explainability of the termination. In this Master's thesis, we focus on efficient modeling for the detection task and how it can be improved. **We implement a model based on deep anomaly detection methods** to predict contract termination with features **based on client knowledge**. More precisely:

1. We implemented unsupervised, semi-supervised, and weakly-supervised state-of-the-art models. We showed that weakly-supervised outperformed the other models and demonstrated that this is the type of model that best fits our task and set-up.
2. In the context of our task, we explored to what extent the architecture of the FEAWAD anomaly detection model can improve the performance of a classical classifier. We have developed AD_MLP, a model that builds on part of the FEAWAD architecture and performs better than other baseline models. The models used for comparison are Logistic Regression, a GLM that represents a well-established standard in the insurance field, and a classical multi-layer perceptron. Specifically, we compare these models using the AUC-ROC and AUC-PR, as well as the F1 score and its associated recall and precision.
3. We study how these models behave when the proportion of class 1 varies from very low to more typical levels, and demonstrate (1) AD_MLP performs better than compared models regardless of the proportion of class-1. (2) anomaly detection features ensure a high level of recall regardless of the proportion.

1 Introduction

1.1 Context and motivation

Customer satisfaction has long been one of the strongest levers of marketing performance. For insurers, the commercial risk of dissatisfaction is churn: the early termination of policies that erodes premium income, weakens cross-sell potential, and raises acquisition costs. As a result, insurers not only monitor satisfaction and retention metrics; they increasingly seek to anticipate termination in order to intervene proactively.

Today, AXA operates in more than 60 countries, serving roughly 94 million customers with a workforce of about 147,000. Within the Group, AXA France is one of the most significant subsidiaries, accounting for roughly 29 percent of Group turnover, and offering a broad portfolio, property and casualty, life and savings, retirement, health, and asset management, through a nationwide network of over 3,000 agencies and 8,300 advisers. In Figure 1, we can see AXA France turnover repartition and observe how important is the casualty insurance for turnover. This footprint makes customer knowledge not just a strategic asset but an operational necessity for day-to-day decisions in sales, service, pricing, and retention.

The abundance of client data creates an imperative: Develop robust decision-support tools capable of extracting signal from mixed, high-dimensional, and often sparse data while meeting increasing regulatory expectations around governance and consumer outcomes.

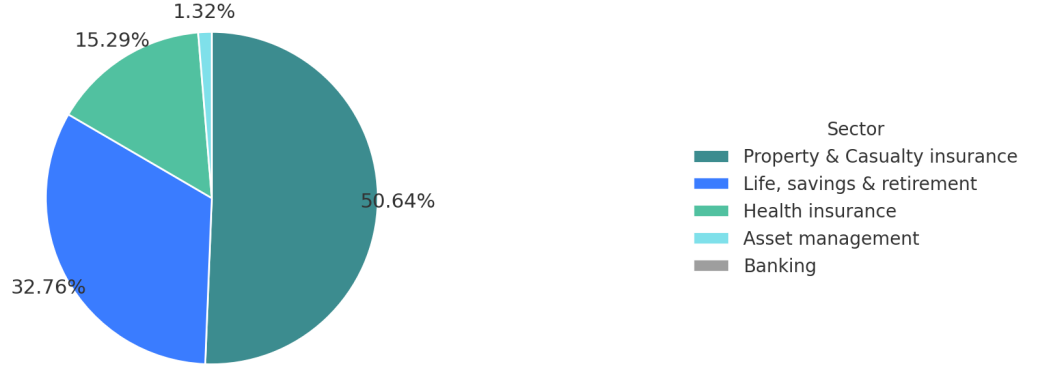


Figure 1: Distribution of Axa France turnover by sector

1.2 Problem statement

This master’s thesis is motivated by these business and methodological needs. It aims to take advantage of AXA’s client data to develop a model that predicts the early *contract termination* (churn) of motor insurance policies i.e. detection in a short period of time after the subscription of a contract. The target flags whether the contract will be terminated within the next short period of time for a given date. From a *ML* point of view, the challenge is twofold:

1. **Severe class imbalance and rare events.** Short-term terminations form a small minority of records. In such imbalanced regimes, ranking metrics (e.g., PR-AUC) and cost-sensitive objectives that emphasize *recall* are more informative than accuracy or ROC-AUC. we must maximize the detection of terminations and minimize non-detection.
2. **Mixed categorical–numerical features.** In order to leverage the client features which are both categorical and numerical, we need to adopt architectures that enable models to learn expressive, well-calibrated representations across feature types.

Concretely, we study the following question: **In the context of churn prediction, to what extent anomaly detection methods enable a model to perform better on AUC and recall-centric metrics compared to GLMs and classical neural network classifiers ?**

After introducing the state of the art in Section 2 of anomaly detection and GLMs, we will give insights about the dataset used and how it was built in Section 3.1.2. In Section 6, we make a benchmark of different type of deep AD (Anomaly Detection) models. We will next design an anomaly detection-based model and study its strengths compared to baselines in Section 3.2.2. Finally, we will study the impact of the variation in class proportion on the model’s performances in 3.2.3.

2 State of the art

2.1 Generalized Linear Models

2.1.1 Definitions and principles

Generalized Linear Models (GLMs) provide a unified framework that extends classical linear regression to accommodate a broader range of data distributions and relationships. The GLM concept was introduced by J. A. Nelder and R. W. M. Wedderburn in 1972 [1] as a generalization of ordinary least squares regression, using maximum-likelihood estimation to handle models where observations follow an exponential-family distribution and the effects of predictors become linear after a suitable transformation (See subsection 3). This innovation brought together various previously separate methods (e.g., logistic regression for binary data, log-linear models for counts) under one theoretical umbrella, allowing normal, binomial, Poisson, gamma and other distributions to be modeled with a consistent approach.

GLMs have since become a key methodology in fields like insurance, where they serve as a strong baseline for many important outcomes (claim counts, policy lapses, risk modeling) that violate the assumptions of ordinary linear models.

Classical linear regression relies on restrictive assumptions: normally-distributed errors, homoscedasticity (constant variance) and a direct linear relationship between the target variable Y (e.g. the response variable) and the features (e.g. the predictor variables). GLMs relax all of these assumptions:

1. Instead of requiring normal residuals, a GLM allows the response Y to follow any distribution from the *exponential family* (e.g., binomial for binary data, Poisson for counts, etc.), which vastly expands the types of outcome variables that can be modeled.
2. GLMs do not assume constant variance; the variance is generally a function of the mean (as determined by the chosen distribution), permitting heteroscedasticity to be naturally handled.
3. GLMs introduce a *link function* that relates the expected value of Y to the linear combination of predictors, thereby accommodating nonlinear relationships between Y and the predictors while still maintaining linearity on the transformed scale.

2.1.2 Core components

Formally, a GLM is defined by three core components:

1. **Random component (Error distribution):** The distribution of the response variable Y_i is assumed to belong to the exponential family (Normal, Binomial, Poisson, Gamma, etc.). That is, for some canonical parameter θ (depending on the predictors) and dispersion ϕ , the density or mass function can be written in the form

$$\pi(z; \theta, \phi) = \exp\left(\frac{z\theta - b(\theta)}{a(\phi)} + c(z, \phi)\right),$$

where:

- z is the target variable (e.g. response variable).
- θ is the *canonical parameter*, which shifts the distribution's mean.
- ϕ is the *dispersion parameter*, governing the scale of the variance.
- $a(\phi)$ is called the *scale function*, It defines how "large" or "tight" the distribution is.
- $b(\theta)$ is called the *cumulant function* (ensuring normalization via the log-partition term). Its derivatives with respect to θ yield the mean and variance.

- $c(z, \phi)$ is an *additional normalization term* that ensures f integrates (or sums) to 1.

We denote $\mu_i = E[Z_i]$ as the mean of the response. The choice of distribution is guided by the data type (e.g. binary, count, continuous) and determines features like the variance $V(Z_i)$ as a function of μ_i . *Example:* $V(Z_i) = \mu_i$ in a *Poisson model*.

In practice, model parameters are typically estimated by maximizing the likelihood of the chosen family using iterative algorithms such as iteratively reweighted least squares or gradient descent.

- 2. Systematic component (Linear predictor):** The GLM includes a linear predictor y_i that is a linear combination of the model's explanatory variables. In other words, for an observation i with features $x_{i1}, x_{i2}, \dots, x_{ip}$, the model defines

$$y_i = \beta_0 + \sum_{j=1}^p \beta_j x_{ij} = \mathbf{x}_i^\top \boldsymbol{\beta}.$$

e.g.

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} = \mathbf{x}_i^\top \boldsymbol{\beta},$$

where:

- y is the linear predictor.
- $\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_p)$ are the coefficients to be estimated.

This linear predictor η_i is the “systematic” part of the model that captures the combined effect of predictors on a transformed scale. It is analogous to the $X\beta$ term in ordinary linear regression.

- 3. Link function:** The link function f connects the parameter θ of the distribution of z with the y 's of the linear model.

$$\theta = f(y)$$

The canonical example in ordinary regression is the identity link $f(y) = y$, but GLMs allow many other links appropriate to the distribution of Z allowing to model non-linear relationships within a linear modeling framework. A link is called canonical when it sets the linear predictor equal to the natural parameter of the exponential-family distribution for Z . The link function is often chosen to be the *canonical link* for the distribution (e.g. logit for binomial, log for Poisson) because this simplifies theoretical properties and interpretation, but non-canonical links can also be used if appropriate.

When Z is normally distributed with mean θ and variance σ^2 and when $\theta = Y$, we have ordinary linear models with Normal errors.

2.1.3 Classification and Regression

One of the strengths of GLMs is their ability to handle both traditional regression tasks and classification tasks (both binary and multiclass) within the same unified framework, simply by selecting an appropriate distribution and link function. We will now illustrate this with two famous examples from the field of insurance.

2.1.3.1 Example 1: Logistic Regression for client churn prediction A more detailed example can be found in [4]. Logistic regression is a GLM used for binary classification problems. Here the response Z is assumed to be binary (0/1, e.g. “contract not terminated” vs “contract terminated”), and we model Z using a binomial distribution with parameter $\theta = P(Z = 1)$. The canonical link is the logit function:

$$f(\theta) = \log\left(\frac{\theta}{1 - \theta}\right) = \mathbf{x}^\top \boldsymbol{\beta}.$$

$$\theta_i = \Pr(Z_i = 1)$$

$$\theta_i = \frac{1}{1 + \exp(-\mathbf{x}_i^T \beta)}$$

One can model the lapse probability of a life insurance policy as a function of policyholder attributes and economic factors. Logistic regression has historically been the de facto standard for such classification tasks in insurance (often termed “churn”), offering interpretable coefficients (in terms of log-odds) for each risk factor.

2.1.3.2 Example 2: Poisson Regression for the expected number of claims Poisson regression is a GLM tailored for count data. It assumes that the response Z is a count (taking values $0, 1, 2, \dots$) following a Poisson distribution with mean $\theta = E[Z]$. The typical link is the log link:

$$f(\theta) = \log(\theta) = \mathbf{x}^T \beta.$$

$$\theta_i = \exp(\mathbf{x}_i^T \beta)$$

which ensures the predicted mean is non-negative. More details in [5].

In automobile insurance pricing, one can use the Poisson Regression to model the expected number of claims a driver will file in a year based on characteristics like age, driving record, and vehicle type. Poisson regression provides a straightforward and interpretable way to relate risk factors to claim frequency, and it has long been a foundational tool in actuarial pricing models.

2.2 Anomaly Detection

2.2.1 Definitions and principles

2.2.1.1 Anomaly definition and intrinsic properties

An anomaly (also known as an outlier or a novelty) is typically defined as a data instance that deviates significantly from the majority of the data, i.e. it does not conform to the expected pattern of the dataset. In the context of insurance contracts, anomalies might correspond to contracts or customer behaviors that are highly unusual compared to the normal portfolio patterns (*or example, a policy with an extremely high claim frequency or an unexpectedly early termination*).

First of all, anomalies are mostly characterized by their unusual and unpredictable behavior, which introduces a high factor of unknownness. Typically, anomalies are not enumerable in advance, or even worst, they may already be introduced in the known data without our knowledge. This leads to them having heterogeneous semantics and features, with the only thing they have in common being their rarity. Usually, we are faced with severe class imbalance because anomalies are rare by nature but also because they are difficult or impossible to collect (i.e., to label). In most of the use cases, misclassifying an anomaly is very costly. For example, predicting that a client will not end his contract when he actually will is more serious for the company than the opposite. In medical image analysis such as cancer detection, one faces the same problem.

Finally, anomalies are characterized by a high diversity of types. We can define three main types of anomalies:

1. **Point anomalies** are data instances which are anomalous as individuals with respect to other individual instances.
2. **Conditional anomalies** are instances that are abnormal in a specific context but are normal otherwise. *For example, a financial index that drops after bad political news may not be considered an anomaly, but it would be in another political context.*
3. **Group anomalies** are subset of data instances which may not be anomalies as individuals but are anomalous as a whole. *For example in a database, a line indicates that a client terminates a contract x at time step t and a second line indicates that the same client subscribes to a contract x at time step $t+1$. We could consider these two events together as one anomaly because it is impossible to terminate a contract that is not created yet even though these two lines are individually normal.*

2.2.1.2 Key challenges

Anomaly detection presents several key challenges

- **Low anomaly detection rate** is one of the most important and yet difficult challenges in many fields for the significant expense of failing to spotting anomalies (See examples in subsection 2.2.1.1). Indeed, many important instances are wrongly reported as abnormal while true yet sophisticated anomalies are missed.
- **Detection in high-dimensional:** Anomalies often display clear abnormal patterns in low-dimensional spaces but can become hidden or difficult to detect in high-dimensional settings a long-standing challenge in anomaly detection. If we use a reduced lower-dimensional space, it is hard to guarantee proper information on the features due to the aforementioned properties.
- **Data-efficient learning of normality/abnormality:** Collecting large-scale labeled anomalies is mostly impossible and therefore, fully supervised anomaly detection is impractical. A lot of research efforts have been focused on unsupervised learning as it does not require any labeled data but make strong assumptions on the distribution of anomalies. Semi-supervised and weakly-

supervised learning are other research lines that cover the case of incomplete/partial classes which we will discuss later. The challenge here are 1) how to learn expressive normality/abnormality representations with small amount of labeled anomalies and 2) how to build model that are able to generalize to novel anomalies.

- **Noise-resilient detection:** The majority of the semi/weakly-supervised assume that the training data is clean. This implies two type of issues. First, the noise can be either mislabeled data or unlabeled anomalies. Second, anomalies can be confused with data which is not clean and therefore deviates from the clean data thus models might confuse them. In such case, unsupervised methods may be the solution but they fail to leverage the available labeled data.
- **Complex (group of conditional) anomalies detection** methods are less developed than point anomalies ones.
- **Anomaly explanation** derivation from specific detection methods is still a largely unsolved problem. In many safety-critical domains there may be some major risks if anomaly detection models are directly used as black-box models. For example, the rare data instances reported as anomalies may lead to possible algorithmic bias against the minority groups presented in the data, such as under-represented groups in fraud detection and crime detection systems. An effective approach to mitigating this type of risk is to use anomaly explanation algorithms that provide clear indications as to why a specific data instance has been identified as an anomaly. Human experts can then investigate and correct any bias.

Insurances have also related challenges that will be faces by such methods:

- **Mixed-type features:** Insurance data typically contains a mix of numerical features (e.g., ages, premium amounts) and categorical features (e.g., policy type, region, demographic informations). Many anomaly detection algorithms (especially distance-based ones) operate in a vector space and rely on distance or density computations that are straightforward in purely numeric feature spaces. Simple one-hot encoding may results in poorer features representation and therefore in poorer models performance.
- **Concept drift:** The notion of normal behavior in insurance can evolve over time. Customer behaviors, market conditions, or regulatory environments may shift, meaning that what was once an outlier could later become common (and vice versa). This phenomenon, known as concept drift, causes the statistical properties of “normal” data to change over time, potentially leading a model to misclassify newly emerging normal patterns as anomalies or overlook new kinds of anomalies.

2.2.1.3 Modeling approaches

Let’s take a look at the modeling principles for anomaly detections before we dive into the methods in details. Table 1 summarises the key characteristics; the following paragraphs provide the details.

In the anomaly-detection literature three broad learning paradigms are commonly distinguished, namely *unsupervised*, *semi-supervised* and *weakly-supervised* approaches. The fundamental difference between them lies in the type and the amount of prior informations that are available at training time and, consequently, in the statistical principles they rely on to separate normal from abnormal behaviour. But they also differ from each other in the manner they make use of the available known anomalies. Overall, all the methods are based on the one thing we know for certain about the data: that they deviate significantly. That is, the models built try to learn a compact and dense representation of normal data in order to easily identify anomalies that deviate from it.

Types of learning Unsupervised models assume that the training set consists of predominantly normal observation. Therefore, it makes strong assumptions on the data distribution but on the other

hand enable us to get rid of problem about the small amount of labeled data. *For example, statistical density estimators (e.g. GMM) approximate the support of the data-generating distribution. Points falling into low-density regions are flagged as anomalies. Reconstruction-based models such as auto-encoders or (Variational) GANs learn a low-dimensional manifold of the data. Large reconstruction error $\|\mathbf{x} - \hat{\mathbf{x}}\|$ indicates abnormality.*

Semi-supervision is used when only a subset of the labeled are known. In the context of anomaly detection, the models only rely on the normal data and anomalies are assumed unavailable and not used during training. The detector tries to learn a strong representation of the normal data and treat any sample outside that is different as an outlier. This type of models are preferred when a reference set of normal behavior exists.

Weakly-supervision formally means that we only know a subset of the labels and we do not fully trust their classification. In the context of anomaly detection, we make the assumption that a very small quantity of relatively trustworthy anomalies are available. Because both classes are partially observed, the task can be phrased as "learning an anomaly score $s\theta(x)$ that ranks labeled outliers above unlabeled presumably normal samples". In practice, we label to 1 all the anomalies and to 0 every other unlabeled instances. In order to deal with the lack of generalization (which is due to partially known classes), these methods introduce a "contamination rate". It consists of randomly switching a small amount of labels to the opposite class as a way to simulate the potential unlabeled anomalies or mislabeled normal instances in the training set. The "weakly" supervision comes from the assumption that unlabeled data are not fully trustworthy. Weakly-supervised models prevent overfitting by keeping normal scores tightly clustered around a fixed, data-independent reference distribution, so the abundant unlabeled data act as continual "pseudo-labels" that pull the model back toward that prior. The few labelled anomalies merely have to exceed a preset margin via pairwise ranking constraints, preventing the network from endlessly reshaping its score space and thus curbing overfitting despite severe class imbalance.

These models are preferred when at least a handful of true outliers can be curated, allowing the model to align its score with practitioner interest while remaining data-efficient.

Training method: In weakly and semi supervised methods, one common way to proceed is to train or pre-train models with only normal samples and thus adresssing the problem as a one-class problem. Therefore, the model has a strong knowledge about normal data and can identify any new instance that deviates from the learned ones.

Overall, the three paradigms are *complementary*. In this thesis we benchmark representatives of each family and later exploit the weak supervision setting to maximise recall under severe class imbalance in the task of contract termination detection.

Table 1: Learning paradigms for anomaly detection and their underlying principles.

Paradigm	Available labels	Underlying principle / typical objective
Unsupervised	none	Density, distance or reconstruction based characterisation of <i>normality</i> ; anomalies are samples that <i>deviate significantly</i> from the learned data representation.
Semi-supervised	only normal samples labelled	One-class description of normal data; anomalies violate the learned <i>compact support</i> of the nominal class.
Weakly-supervised	few anomaly labels in addition to abundant unlabeled data	End-to-end optimisation of an <i>anomaly scoring function</i> guided by the labelled outliers, often regularised by a prior on normal scores.

2.2.2 Classic models

We will now review the three main anomaly detection models that rely on "classic" machine learning, namely Isolation Forest, One-Class Support Vector Machine, and k-Nearest Neighbors based approach. Unlike supervised classifiers, these methods operate in an unsupervised or one-class setting.

2.2.2.1 Isolation Forest (iForest)

Intuition: Isolation Forest detects anomalies by randomly isolating data points using binary decision trees. The key idea is that outlier instances are easier to separate from the rest of the data because they exhibit attribute values that are markedly different or sparse compared to the majority. In each tree, features and split points are chosen at random. An anomalous instance will likely be isolated by a few random splits, ending up at a shallow depth of the tree. In contrast, normal contracts that share common characteristics require more splits to isolate, thus producing deeper tree paths. Intuition is depicted in Figure 2.

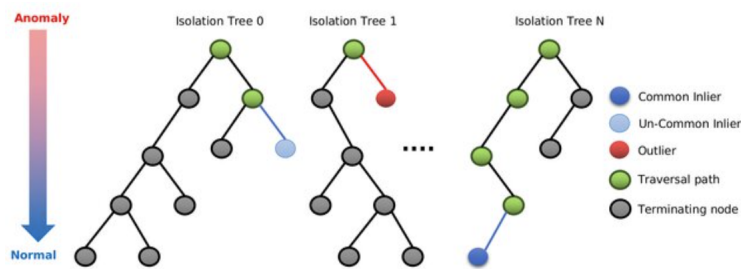


Figure 2: Illustration of iForest [9]

Formal Description: Formally, an Isolation Forest [8] is an ensemble of t isolation trees built on portions of the data (usually sub-sampled from the main dataset). Each tree is grown by recursively partitioning the data: at each node, a feature is selected uniformly at random and a split value is chosen uniformly between the feature's minimum and maximum values in that node's subset. Splitting continues until each instance is isolated in its own leaf or until a maximum tree depth is reached. The length of the path $h(x)$ from the root to the leaf where an instance x is isolated is recorded for each

tree. An anomaly score is then defined based on the average path length $E[h(x)]$ over all trees. A common scoring function is:

$$s(x) = 2^{-\frac{E[h(x)]}{c(m)}},$$

where:

- m is the subsample size used to build each tree.
- $E[h(x)]$ is the average of $h(x)$ from a collection of iTrees.
- $c(m)$ the average $h(x)$ over the subsamples (a normalization constant).

This score $s(x)$ is in the range $[0, 1]$, with values close to 1 indicating points that have unusually short average paths (most likely to be anomalies), while values around 0 indicate points with long paths (likely normal).

In practice, one selects a threshold on $s(x)$ (or equivalently on $E[h(x)]$) to decide which data points are flagged as anomalies.

Importantly, the Isolation Forest's complexity scales approximately linearly with the number of instances (due to the use of random splits and often subsampling), making it feasible to apply on large insurance datasets.

Strengths and Limitations: iForest has the following advantages:

1. It does not assume any specific data distribution and does not rely on distance measures, which can be useful given a set of mixed attributes (numeric and categorical).
2. It is computationally efficient, parallelizable, and highly scalable.
3. The model naturally accommodates high-dimensional data and don't rely on a specific distance or kernel choice.

However, the model has significant limitations:

1. Isolation Forest may struggle to detect anomalies that are subtle or lie in regions of data that are only slightly sparser than normal. If instances do not exhibit extreme feature values or if they form a small cluster of their own (sharing characteristics with each other but not with the majority), they might not be isolated much sooner than typical instances.
2. One must be careful about the threshold he chooses as it may significantly impact the model performance.
3. Because the algorithm is random, its outcome can vary slightly between runs.

2.2.2.2 One-Class SVM

Intuition: One-Class Support Vector Machine (OC-SVM) identifies anomalies by learning a boundary that encapsulates the normal data points in the feature space, classifying anything outside this boundary as an outlier. Intuitively, OC-SVM attempts to carve out a region (often envisioned as a contour or hypersphere in feature space) that encloses the majority of the training instances (in our case, contracts that are presumed to continue normally). In essence, the method tries to separate the normal contracts from the origin with a maximal margin. One can imagine it as finding the tightest envelope around the normal data: anything that sticks out of this envelope is flagged as a novelty or outlier. During training, only normal instances are considered in order to learn the envelope aforementioned and during prediction, the model classify any instance that falls outside of it as an anomaly. Intuition is depicted in Figure 3 and 4, we can see an illustration of how data can be mapped to a feature space in order to differentiate anomalies and variables such as ξ offer flexibility with soft margin.

Formal Description: Formally, One-Class SVM solves an optimization problem to find a decision function $f(x)$ whose positive level set corresponds to the region containing most data points. A common formulation introduces a parameter $\nu \in [0, 1]$ that controls the trade-off between capturing most points in the “normal” region and allowing a certain fraction to be outliers. One convenient way to express the OC-SVM problem is:

$$\begin{aligned} \min_{w, \rho, \{\xi_i\}} \quad & \frac{1}{2} \|w\|^2 + \frac{1}{\nu n} \sum_{i=1}^n \xi_i - \rho, \\ \text{subject to} \quad & w^\top \Phi(x_i) \geq \rho - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n, \end{aligned} \quad (1)$$

where:

- $\Phi(x)$ is a (possibly non-linear) feature mapping induced by a kernel.
- w is a weight vector.
- ν the aforementioned trade-off variable.
- ρ is a bias term.
- ξ_i are slack variables introduced to soften the margin.

In this formulation, the term $\frac{1}{2} \|w\|^2$ is minimized to maximize the margin (keeping the boundary away from the origin), while ρ is effectively the offset that adjusts how far the boundary is from the origin. The constraints enforce that most data points x_i lie above the decision boundary $w^\top \Phi(x) = \rho$ (with some slack ξ_i allowed for those that might lie outside).

The decision function derived from this optimization is $f(x) = w^\top \Phi(x) - \rho$. At prediction time, the sign of $f(x)$ determines novelty: if $f(x) < 0$, the point x lies outside the learned region (flagged as anomaly), whereas $f(x) \geq 0$ means x is within the estimated support of normal data. By choosing an appropriate kernel, OC-SVM can learn a non-linear boundary that wraps around the normal instances.

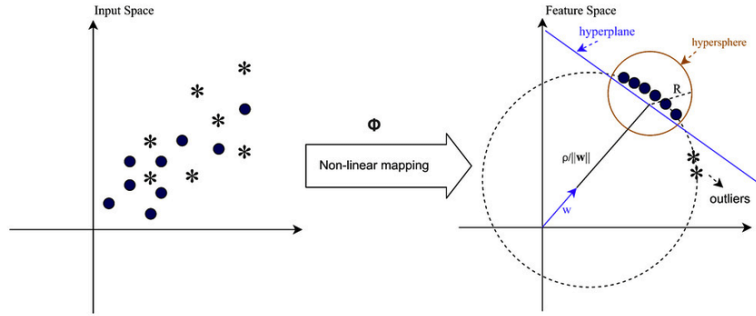


Figure 3: Illustration of one class SVM problem

Strengths and Limitations: OC-SVM offers the following advantages:

1. The model is able to capture non-linear relationships thanks to use of the kernel trick. Because the kernel is not unique, one can adapt to several use cases by adapting it.
2. variables such as μ and ξ offer a significant flexibility to the model. Indeed, the trade-off introduced by μ enables the model to increase its sensitivity to anomaly detection (For instance, aligning with an expected churn rate if known in the case of contract terminations detection). ξ offers a higher robustness to noise.

However, OC-SVM comes with notable limitations.

1. It is computationally intensive for large n : training complexity is at least quadratic (or worse) in the number of instances, so scaling to tens or hundreds of thousands of contracts may be

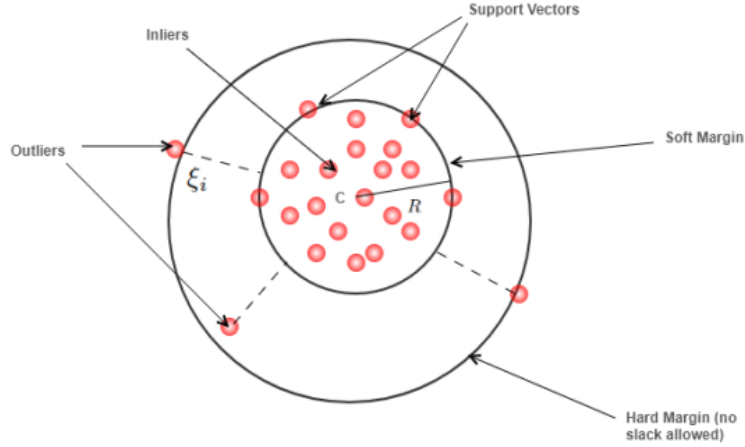


Figure 4: Illustration of one class SVM problem

infeasible without approximation. This is a serious concern, especially for insurance use cases where customer and contracts databases are often very large.

2. The method is sensitive to the hyperparameters. An inappropriate kernel or a poorly tuned kernel parameter can lead to a boundary that either hugs the data too tightly (flagging too many false positives) or is too loose (missing anomalies). Similarly, setting ν requires care: if early contract terminations are, say, around 5 percents of the data, one might start with $\nu = 0.05$, but the optimal value might differ; too high a ν would wrongly label a large portion of normal contracts as outliers, while too low a ν might ignore legitimate anomalies.
3. If the "normal" data actually consists of multiple clusters or modes, a single OC-SVM decision boundary might not capture all of them well. In practice, one might need to train separate OC-SVM models per segment or use a kernel that can handle multi-modality, but that complicates the approach.
4. Like most kernel methods, acts as a black box in terms of interpretability. Indeed, the underlying transformations handled by the kernel make it difficult to understand why an instance is or is not in the good region in the feature space.

2.2.2.3 k-NN and Local Outlier Factor

Intuition: The k-Nearest Neighbors (k-NN) approach to anomaly detection is based on the idea that normal instances tend to be near other similar instances, whereas anomalies are often isolated from their peers. In a simple k-NN outlier detection scheme, one can assign an anomaly score to a data point based on its distance to its k -th nearest neighbor (or some aggregate of distances to the k nearest neighbors).

In order to handle the case of some instances are "far" (in term of the chosen distance) from their neighbours but are not anomalies, we can introduce the notion of Local Outlier Factor (LOF). It extends the idea by considering the relative density of a point compared to the density of its neighborhood. LOF asks not just "how far is x from others?" but also "is x in a region significantly less dense than the regions where its neighbors lie?". In other words, an instance might not have an extreme distance to its nearest neighbors, but if those neighbors are very close to each other (forming a tight cluster) and x is somewhat loosely connected, then x 's neighborhood is sparse relative to theirs, making x a local outlier. This concept allows LOF to detect points that are outliers in their local context, which a basic k-NN distance alone would miss. Intuition is depicted in Figure 5, we can see

an example of the outlier scores based on the density around the points which allow to identify local outliers.

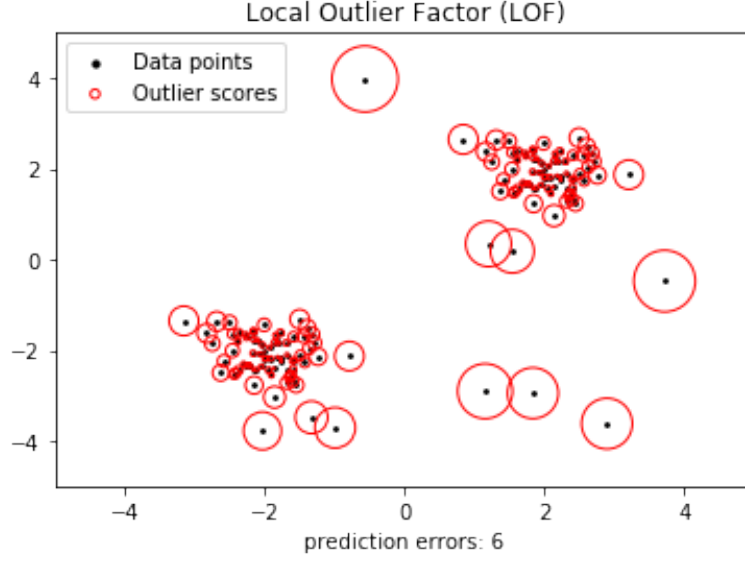


Figure 5: Illustration of k-NN with Local Outlier Factor

Formal Description: For each point x we define $D_k(x)$ as the distance from x to its k -th nearest neighbor. This k -distance can serve as a simple anomaly score or threshold: a larger $D_k(x)$ implies x is more isolated. Building on this, the Local Outlier Factor introduces a more nuanced calculation. First, define the reachability distance from x to a neighbor y :

$$\text{reach}_k(x, y) = \max\{D_k(y), d(x, y)\},$$

where $d(x, y)$ is the distance between points x and y . This means that if x is extremely close to y , we use at least y 's k -distance as a floor, which prevents one very close neighbor from artificially inflating the density estimate. Next, we compute the local reachability density (LRD) of x as the inverse of the average reachability distance of x from its k nearest neighbors $N_k(x)$:

$$\text{lrd}_k(x) = \left(\frac{1}{|N_k(x)|} \sum_{y \in N_k(x)} \text{reach}_k(x, y) \right)^{-1}.$$

Intuitively, $\text{lrd}_k(x)$ will be high if x can reach its neighbors with only small distances (i.e. x is in a dense area), and low if x is in a sparse area relative to its neighbors. Finally, the local outlier factor is defined by comparing x 's density to the densities of its neighbors:

$$\text{LOF}_k(x) = \frac{1}{|N_k(x)|} \sum_{y \in N_k(x)} \frac{\text{lrd}_k(y)}{\text{lrd}_k(x)}.$$

In essence, $\text{LOF}_k(x)$ is the average ratio of the neighbors' density to x 's density.

- If $\text{LOF}_k(x) > 1$, x has a much lower density than its neighbors which indicates that it is most likely to be an anomaly.
- $\text{LOF}_k(x) \approx 1$ means that x is not locally an outlier.
- $\text{LOF}_k(x) \approx < 1$ typically doesn't happen unless there are boundary effects or noise. However, it would mean that x is in a denser region compared to its neighbors.

The LOF method thus produces a continuous anomaly score rather than a binary flag. About the choice of k : a smaller k captures local structure very tightly, while a larger k makes the density comparison more global.

Strengths and Limitations: k -NN with LOF has the following advantages:

1. It makes no assumptions about the underlying distribution of normal instances but instead relies purely on the geometry of the data in feature space.
2. It handles complex datasets with multiple clusters or irregular shapes.
3. The concept of a "distance to neighbors" can be somewhat interpretable. We can compare an outlier with its nearest neighbor to understand what it is compared against.

And has the following limitations:

1. It doesn't scale well. The training complexity is $O(n^2)$. Indeed, to evaluate a data point, we need to compare it to every single point in the dataset.
2. Because it is a metric-based method, it suffers highly from the curse of dimensionality. Furthermore, distance metrics like euclidean norm may not be very efficient for one-hot encoded categorical data which further complicates the choice of distance.
3. It is very sensitive to the choice of k .

2.2.3 Deep Anomaly Detection

2.2.3.1 FEAAD: Features Encoding for Weakly-supervised Anomaly Detection

Intuition: Deep weakly supervised models are composed by an "anomaly score generator" that essentially consists of a MLP network. What makes this MLP efficient for anomaly detection is what it is fed with and the objective function on which it is optimized. Indeed, the objective function is built so that it can penalize as much as possible the instances that deviate from the majority.

FEAWAD [11] is built on DevNet [10]. DevNet is a deep anomaly detector that learns the anomaly score directly end-to-end (instead of learning features first and scoring later like other methods often do). It uses a neural backbone $\psi(x)$ followed by a single linear output unit η to produce a scalar score, and anchors "normal" scores with a Gaussian prior to get a reference mean/variance. Training minimizes a Z-score deviation loss: normals are pulled toward the reference, while labeled anomalies are pushed above a margin.

FEAWAD enhances the anomaly score generator and simplifies the objective function with feature representation variables learned by an autoencoder and thus overperforms the state of the art for tabular anomaly detection.

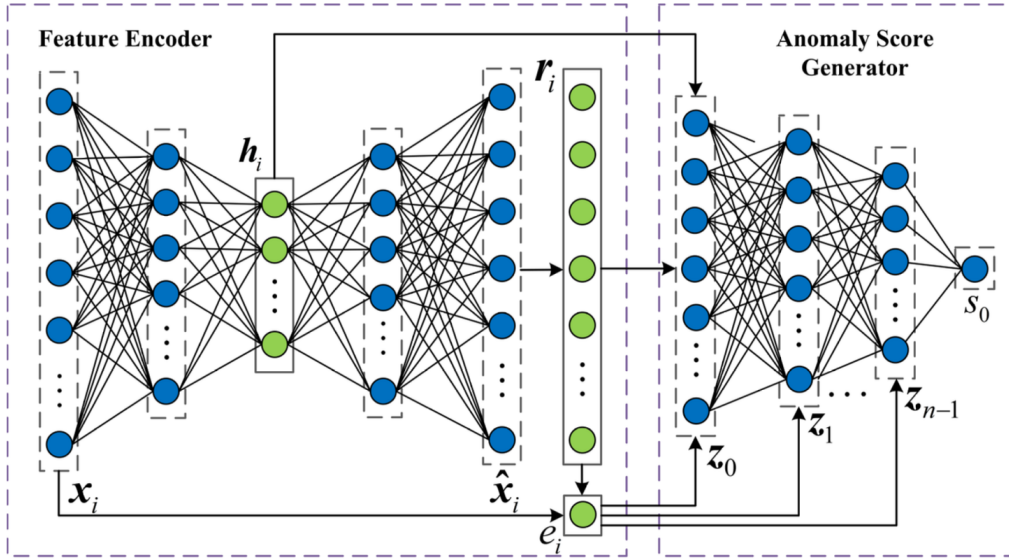


Figure 6: The architecture of FEAAD model

Architecture: The architecture of the model is depicted in 6. The objective function is fed with the following feature representations that are built by the model:

- **Latent representation** h
- **Reconstruction error** $e = \|x - \hat{x}\|^2$
- **Residual** $r = \frac{\hat{x}_i - x_i}{\|\hat{x}_i - x_i\|_2}$.

The **final objective** is:

$$\mathcal{L}(\Theta_e, \Theta_g) = \mathcal{L}_d(\Theta_e, \Theta_g) + \lambda \mathcal{L}_e(\Theta_e)$$

with:

- The **deviation loss** $\mathcal{L}_d = \sum_i (1 - y_i) |\phi(x_i)| + y_i \max(0, a_0 - \phi(x_i))$
- The **reconstruction loss**: $\mathcal{L}_e = \sum_i (1 - y_i) e_i + y_i \max(0, a_0 - e_i)$

- $\phi(x_i) = s_i$ the anomaly score of the instance i .
- The parameter of the autoencoder Θ_e
- The parameters of the anomaly score generator Θ_g

Training process: The training is divided into 1) the pre-training of the autoencoder but only on normal data and 2) an end-to-end optimization with batches composed of half normal instances and half anomalies. Training algorithm is depicted in 7.

Pre-training aligns with the practice of other models that focus on learning a strong representation of normal data in order to discriminate anomalies more easily. The aforementioned batch composition during the end-to-end optimization is a strategy to fight class imbalance to ensure that both of the loss terms contribute i.e. the network learns to both pull normals to the reference and push anomalies above the margin at the same time.

Algorithm 1 Network Training Procedure

Stage 1: Pre-training
Input: $X^{\text{unlabeled}} \in \mathbb{R}^m$
Output: $\psi(\cdot; \Theta_e) : \mathbb{R}^m \mapsto \mathbb{R}^m$

- 1: Randomly initialize Θ_e
- 2: **repeat**
- 3: Randomly sample one batch of unlabeled data.
- 4: Compute the loss $\mathcal{L}_u$ and update weight matrices of $\psi(\cdot; \Theta_e)$
- 5: **until** converge
- 6: **return** $\psi(\cdot; \Theta_e)$

Stage 2: End-to-end Optimization
Input: $X \in \mathbb{R}^m$
Output: $\phi(\cdot; \Theta_e; \Theta_g) : \mathbb{R}^m \mapsto \mathbb{R}$

- 7: Randomly initialize Θ_g and load Θ_e from the pre-trained network $\psi(\cdot; \Theta_e)$
- 8: **repeat**
- 9: Randomly sample half of samples from unlabeled data and half of samples from labeled anomalies to form a batch
- 10: Compute the joint loss $\mathcal{L}$ and update the network parameters $\{\Theta_e; \Theta_g\}$
- 11: **until** converge
- 12: **return** $\phi(\cdot; \Theta_e; \Theta_g)$

Figure 7: Training algorithm of FEAAD model

2.2.3.2 PReNet: Pairwise Relation prediction Network

Intuition: Semi / weakly supervised anomaly detection methods that are trained using small labeled anomaly examples and large unlabeled data (mostly normal data) have shown largely improved performance over unsupervised methods. Known anomalies only cover a small subspace of all possible anomalies. However, the goal is to detect both seen and unseen anomalies. In classical approaches like DevNet or FEAWAD, the learned representations are tailored to distinguish only the known (i.e. seen anomalies) anomalies from the rest. This biases the model toward recognizing only the specific types of anomalies it has seen during training, and consequently they fail to generalize to those that are not, i.e., new types/classes of anomaly unseen during training, which results in a poor balance between recall and precision. More precisely, they tend to have a poor precision (too many false positives), traducing a too optimist detection of anomalies, and a very high recall (few false negatives). PReNet [12] uses two networks that share the same weights to learn pairwise relation features and anomaly scores by predicting the relation of any two randomly sampled training instances, in which the pairwise relation can be anomaly-anomaly, anomaly-unlabeled, or unlabeled-unlabeled. This helps the network to learn interclass relations instead of just intraclass information. It can be seen as treating data like small group and since unseen anomalies can also have closer patterns to normal data, it enables the model to capture those subtle information. Since unlabeled instances are mostly normal (in our case, fully normal), the relation-prediction enforces a joint learning of anomaly-anomaly, anomaly-normal, and normal-normal pairwise discriminative patterns, respectively. PReNet can then detect any seen/unseen abnormalities that fit the learned pairwise abnormal patterns, or deviate from the normal patterns.

Architecture: As shown in Figure 8, the PReNet architecture consists of two main components that work jointly to learn anomaly scores from data:

- **Anomaly-informed pairing module:** This module generates surrogate training data by randomly pairing instances from unlabeled and labeled data (as well as pairing unlabeled instances with each other). Specifically, instance pairs are created with instances randomly sampled from the small anomaly set $\mathcal{A}$ and the large unlabeled dataset $\mathcal{U}$. A pairwise class label is then assigned to each instance pair, such that $y_{\{a,a\}} = c_1$, $y_{\{a,u\}} = c_2$, $y_{\{u,u\}} = c_3$ and $c_1 > c_2 > c_3 \geq 0$, as long as they provide margins large enough between surrogate classes. By doing so, they efficiently synthesize $\mathcal{A}$ and $\mathcal{U}$ to produce a large labeled dataset $\mathcal{P} = \left\{ \left(\mathbf{x}_i, \mathbf{x}_j, y_{\{\mathbf{x}_i, \mathbf{x}_j\}} \right) \mid \mathbf{x}_i, \mathbf{x}_j \in \mathcal{X} \text{ and } y_{\{\mathbf{x}_i, \mathbf{x}_j\}} \in \mathbb{N} \right\}$. These pairwise labels serve as weak supervision signals. Because most unlabeled samples are normal, these categories effectively capture whether a pair contains an anomaly or not. In the context of churn prediction, the unlabeled samples are all normal. This strategy greatly augments the training data: from only a few anomalies, PReNet synthesizes a large number of labeled pair examples, providing a rich supervisory signal and enforcing learning of both abnormal and normal patterns in a balanced way.
- **Pairwise Relation-based Anomaly Score Learning:** PReNet employs a pairwise anomaly score learner (twin networks with shared weights) $\phi : \mathcal{P} \rightarrow \mathbb{R}$ is then introduced to take $\mathcal{P}$ as input to learn the anomaly scores of instance pairs. Let $\mathcal{Z} \in \mathbb{R}^M$ be an intermediate representation space, they define a two-stream anomaly scoring network $\phi((\cdot, \cdot); \Theta) : \mathcal{P} \rightarrow \mathbb{R}$ as a sequential combination of a feature learner $\psi(\cdot; \Theta_r) : \mathcal{X} \rightarrow \mathcal{Z}$ and an anomaly scoring function $\eta((\cdot, \cdot); \Theta_s) : (\mathcal{Z}, \mathcal{Z}) \rightarrow \mathbb{R}$ where $\Theta = \{\Theta_r, \Theta_s\}$. Specifically, $\psi(\cdot; \Theta_r)$ is a neural *feature learner* with $H \in \mathbb{N}$ hidden layers and the weight parameters Θ_r .

$$\mathbf{z} = \psi(\mathbf{x}; \Theta_r)$$

where $\mathbf{x} \in \mathcal{X}$ and $\mathbf{z} \in \mathcal{Z}$. We define $\eta((\cdot, \cdot); \Theta_s)$ as an *anomaly score learner* that uses a fully connected layer to learn linear pairwise relation features and the anomaly scores, taking the

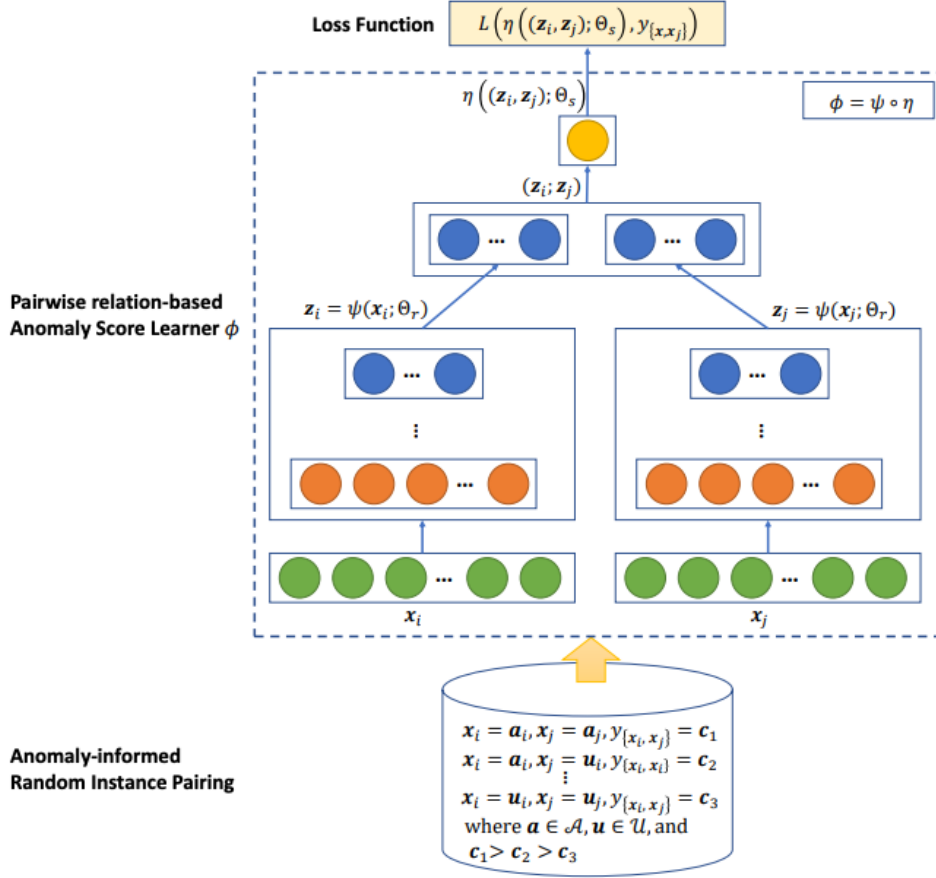


Figure 8: Overview of PReNet architecture. The model uses a twin encoder to extract features h_i, h_j from input pairs, then computes an anomaly score via a relation predictor on the concatenated features. An autoencoder-based reconstruction branch (not shown in detail) provides residual errors e_i to regularize the encoder by modeling normal patterns.

concatenation of the intermediate representations of each pair — $\mathbf{z}_i$ and $\mathbf{z}_j$ — as input:

$$\eta((\mathbf{z}_i, \mathbf{z}_j); \Theta_s) = \sum_{k=1}^M w_k^o z_{ik} + \sum_{l=1}^M w_{M+l}^o z_{jl} + w_{2M+1}^o,$$

where $\mathbf{z} \in \mathcal{Z}$ and $\Theta_s = \{\mathbf{w}^o\}$ in which $\{w_1^o, w_2^o, \dots, w_{2M}^o\}$ are weight parameters and w_{2M+1}^o is a bias term. Thus, $\phi((\cdot, \cdot); \Theta)$ can be formally represented as

$$\phi((\mathbf{x}_i, \mathbf{x}_j); \Theta) = \eta(\psi(\mathbf{x}_i; \Theta_r), \psi(\mathbf{x}_j; \Theta_r); \Theta_s),$$

- **Loss function:** PReNet then uses the pairwise ordinal class labels to optimize the pairwise relation-based anomaly scores. Particularly, it minimizes the difference between the prediction scores and the ordinal labels $y_{\{a,a\}}, y_{\{a,u\}}$ and $y_{\{u,u\}}$. It is equivalent to learning to assign larger prediction scores to the anomaly-related (*i.e.*, anomaly-anomaly and anomaly-unlabeled) instance pairs than the unlabeled-unlabeled pairs. The loss function is defined as below:

$$L(\phi((\mathbf{x}_i, \mathbf{x}_j); \Theta), y_{\{x_i, x_j\}}) = |y_{\{x_i, x_j\}} - \phi((\mathbf{x}_i, \mathbf{x}_j); \Theta)|.$$

The three-class labels $y_{\{a,a\}} = 8$, $y_{\{a,u\}} = 4$ and $y_{\{u,u\}} = 0$ are used by default to enforce a large margin among the anomaly scores of the three types of instance pairs. PReNet also works well with other value assignments as long as there are reasonably large margins among the ordinal labels.

- **Batch sampling strategy:** Lastly, PReNet is trained via:

$$\arg \min_{\Theta} \frac{1}{|\mathcal{B}|} \sum_{(\mathbf{x}_i, \mathbf{x}_j, y_{\{\mathbf{x}_i, \mathbf{x}_j\}}) \in \mathcal{B}} \left| y_{\{\mathbf{x}_i, \mathbf{x}_j\}} - \phi((\mathbf{x}_i, \mathbf{x}_j); \Theta) \right| + \lambda R(\Theta),$$

where $\mathcal{B}$ is a sample batch from $\mathcal{P}$ and $R(\Theta)$ is a regularization term with hyperparameter λ . For each batch, $\frac{|\mathcal{B}|}{2}$ instance pairs are sampled from the $C_{\{u,u\}}$ class and $\frac{|\mathcal{B}|}{4}$ instance pairs are respectively sampled from the $C_{\{a,a\}}$ and $C_{\{a,u\}}$ classes. This is equivalent to oversampling the two anomaly-related classes, $C_{\{a,a\}}$ and $C_{\{a,u\}}$, to avoid bias toward the $C_{\{u,u\}}$ class due to the class imbalance.

Training process: Figure 9 illustrates the training algorithm. Step 1 first extends the data $\mathcal{X}$ into a set of instance pairs with ordinal class labels, $\mathcal{P}$. After a uniform *Glorot* weight initialization in Step 2, PReNet performs stochastic gradient descent (SGD) based optimization to learn Θ in Steps 3–9 and obtains the optimized ϕ in Step 10. Particularly, stratified random sampling is used in Step 5 to ensure the sample balance of the three classes in $\mathcal{B}$, as discussed in Sec. 3.2.2. Step 6 performs the forward propagation of the network and computes the loss. Step 7 then uses the loss to perform gradient descent steps.

Algorithm 1 *Training PReNet*

Input: $\mathcal{X} \in \mathbb{R}^D$ with $\mathcal{X} = \mathcal{U} \cup \mathcal{A}$ and $\emptyset = \mathcal{U} \cap \mathcal{A}$
Output: $\phi : (\mathcal{X}, \mathcal{X}) \rightarrow \mathbb{R}$ - an anomaly score mapping

- 1: $\mathcal{P} \leftarrow$ Augment the training data with $\mathcal{U}$ and $\mathcal{A}$
- 2: Randomly initialize Θ
- 3: **for** $i = 1$ to n_epochs **do**
- 4: **for** $j = 1$ to $n_batches$ **do**
- 5: $\mathcal{B} \leftarrow$ Randomly sample b data instance pairs from $\mathcal{P}$
- 6: $loss \leftarrow \frac{1}{b} \sum_{(\mathbf{x}_i, \mathbf{x}_j, y_{\{\mathbf{x}_i, \mathbf{x}_j\}}) \in \mathcal{B}} \left| y_{\{\mathbf{x}_i, \mathbf{x}_j\}} - \phi((\mathbf{x}_i, \mathbf{x}_j); \Theta) \right| + \lambda R(\Theta)$
- 7: Perform a gradient descent step w.r.t. the parameters in Θ
- 8: **end for**
- 9: **end for**
- 10: **return** ϕ

Figure 9: Training algorithm of PReNet

2.2.3.3 Deep Sad: Sem-supervised Anomaly Detection

Intuition: Deep SAD [14] extends the one-class Deep SVDD model [13] to a semi-supervised setting where a small number of labeled anomalies and/or normals are available. The model $\phi(\mathbf{x}; W)$ is a deep neural network (with L hidden layers and weights $W = \{W^1, \dots, W^L\}$) that maps input $\mathbf{x}$ to a d -dimensional latent. Similar to Deep SVDD, the network is trained to learn a compact representation of the “normal” class by concentrating most data towards a center point c in the latent space. In the unsupervised one-class case, this is achieved by minimizing the volume of a hypersphere centered at c that encloses the data – effectively minimizing $\|\phi(x) - c\|^2$ for all training points. Deep SAD modifies this objective to incorporate m labeled examples $(\tilde{\mathbf{x}}_j, \tilde{y}_j)$ (with labels $\tilde{y} = +1$ for known normal and $\tilde{y} = -1$ for known anomaly) in addition to n unlabeled examples. The aim is to still compactly represent normal data around c , but also push the labeled anomalies away from c in the latent space. In figure 10, we can visualize how a neural network can transform an input into a feature space that allow an efficient grouping of normal data.

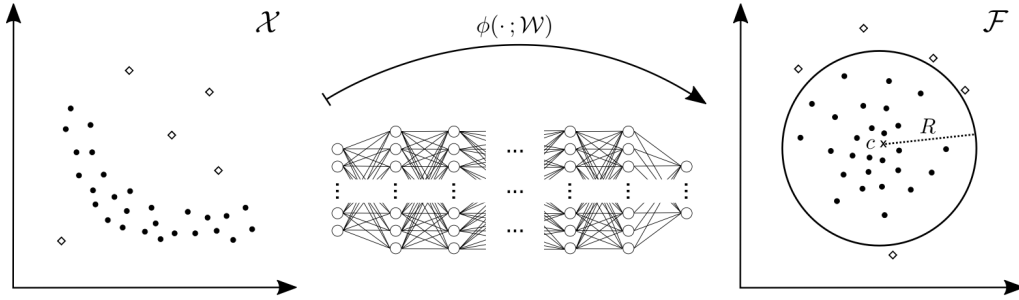


Figure 10: Deep SVDD learns a neural network transformation $\phi(\cdot; W)$ with weights W from input space $\mathcal{X} \subseteq \mathbb{R}^d$ to output space $\mathcal{F} \subseteq \mathbb{R}^p$ that attempts to map most of the data network representations into a hypersphere characterized by center c and radius R of minimum volume. Mappings of normal examples fall within, whereas mappings of anomalies fall outside the hypersphere.

Architecture: The Deep SAD loss function generalizes the Deep SVDD objective by adding a term for labeled data. The overall objective of DeepSAD is:

$$\min_W \frac{1}{n+m} \sum_{i=1}^n \|\phi(x_i; W) - c\|^2 + \frac{\eta}{n+m} \sum_{j=1}^m (\|\phi(\tilde{x}_j; W) - c\|^2)^{\tilde{y}_j} + \frac{\lambda}{2} \sum_{\ell=1}^L \|W^\ell\|_F^2,$$

where:

- n and m are the number of unlabeled and labeled samples respectively.
- $y \in \{-1, 1\}$.
- c is the center of the hypersphere (e.g. initialized as the mean of an initial encoding of normal data).
- η an hyperparameter controlling the relative weight of the labeled terms.
- $\frac{\lambda}{2} \sum_{\ell} \|W^\ell\|_F^2$ is an L_2 weight decay regularization on the network weights (like Deep SVDD).

For unlabeled and labeled-normal data ($\tilde{y} = +1$), the loss term is simply $\|\phi(x) - c\|^2$, which pulls those samples closer to c (just as in one-class SVDD). For labeled anomalies ($\tilde{y} = -1$), the term becomes $(\|\phi(x) - c\|^2)^{-1}$, effectively penalizing the network if an anomaly is too close to c . $\eta > 1$ makes the network focusing more on separating known anomalies, whereas $\eta < 1$ would down-weight them (though in practice $\eta = 1$ is often chosen).

This semi-supervised objective encourages a tight cluster of normal embeddings around c while pushing anomalies to have higher-distance embeddings, aligning with the intuition that anomalies need not be similar to each other but should lie outside the normal data concentration. In the absence of any labeled data, the objective reduces to the one-class Deep SVDD.

Training process: The training process for Deep SAD closely follows the procedure for Deep SVDD, with additional use of labeled data. In practice, one first performs unsupervised ****pre-training**** of the network to initialize a good feature representation. Specifically, an autoencoder is trained on the data (ignoring labels) such that the encoder part has the same architecture as ϕ ; the encoder's weights are then used to initialize ϕ 's weights. After this initialization, the center c is set (for example) to the mean of $\phi(x)$ over a batch of normal data or the entire unlabeled. In [13], authors demonstrate some initial settings that make the model producing trivial solutions for c . Then, they give empirical initial settings that give the best results. These settings are used for Deep SAD. Then the network is trained end-to-end using stochastic gradient descent on the Deep SAD loss, iterating over both unlabeled and labeled samples.

Anomaly detection is performed by evaluating the distance of a test sample's embedding to the center: $s(x) = \|\phi(x) - c\|$. A higher score indicates the sample lies further from the normal cluster and is more likely an anomaly.

2.2.3.4 DIF: Deep Isolation Forest

Intuition: Deep Isolation Forest (DIF) [15] is a high-dimensional, non-linear extension of the Isolation Forest algorithm that. The core idea is to harness the representation power of neural networks to project data into new spaces where simple axis-parallel splits can effectively isolate anomalies that are hard to separate in the original.

In DIF, an "ensemble of random neural networks" is used as the feature mapper: we generate r independent random mappings $\phi_u : \mathbb{R}^D \rightarrow \mathbb{R}^d$ (for $u = 1, \dots, r$), each given by a multi-layer perceptron with randomly initialized weights (and no further training). Applying these r networks to the dataset D yields r different d -dimensional representations $X_u = \phi_u(D)$. On each projected dataset X_u , a set of t isolation trees is built using the standard Isolation Forest procedure (random recursive partitioning). Thus, the overall "deep isolation forest" consists of $T = r \times t$ isolation trees, organized as r groups of t trees, each group operating on one random representation space.

Because each ϕ_u can be a deep nonlinear function, the simple axis-aligned cut $x(j) \leq \eta$ in the X_u space corresponds to a non-linear decision boundary in the original input space. This helps reveal "hard" anomalies that would have required highly complex non-axis-aligned cuts to isolate in the original space.

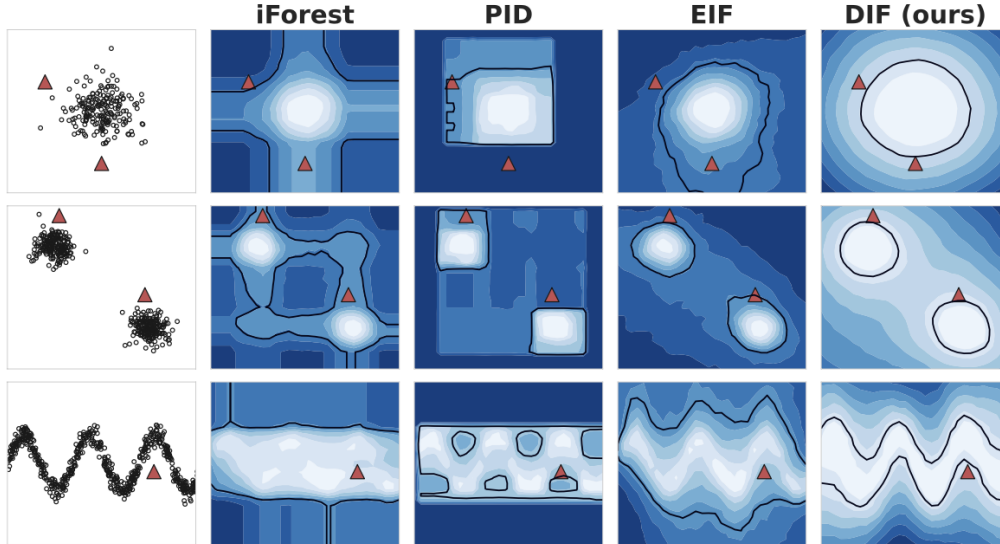


Figure 11: Illustration of the algorithmic bias problem. The three figures on the far left are three synthetic datasets, and the following panels of each dataset are the anomaly score maps produced by iForest and its two state-of-the-art extensions – PID and EIF – and Deep iForest (DIF). Score maps indicate anomaly score distribution in the full data space (deeper colour indicates higher abnormality)

Training process: Unlike typical deep learning models, the networks in DIF are "optimization-free". The weights of each MLP ϕ_u are simply initialized to random values and then fixed. There is therefore no explicit loss function being minimized for the neural networks; the randomness of the projections is what provides diverse partitionings. After generating the random representation ensemble, the only "learning" step is constructing the isolation trees, which is done in a single pass over the data.

To make the random ensemble efficient, DIF introduces a method called "CERE" (Computation-Efficient Representation Ensemble). CERE leverages a clever vectorized weight construction to compute all r network outputs in parallel without running r separate forward passes. Essentially, it generates the weight matrices of each network as a random perturbed copy of a base weight matrix, allowing the ensemble $\{\phi_u\}$ to be collectively evaluated in one batch operation. This significantly

reduces overhead. The time complexity of producing representations r is made comparable to that of a single network forward pass. As a result, DIF scales well even when using many random projections, inheriting the $\mathcal{O}(n \log n)$ overall complexity of Isolation Forest up to constant factors.

Scoring function: For scoring anomalies, DIF employs a customized anomaly scoring function called DEAS (Deviation-Enhanced Anomaly Scoring). In a standard Isolation Forest (2.2.2.1), the anomaly score is derived from the path length of a point in each tree. DIF enhances this by also considering the deviation degree of the feature value to the branching threshold at each split node. We define the averaged deviation degree of x_u in τ_i :

$$g(x_u | \tau_i) = \frac{1}{|p(x_u | \tau_i)|} \sum_{k \in p(x_u | \tau_i)} \left| x_u^{(j_k)} - \eta_k \right|. \quad (2)$$

with:

- x_u be the corresponding representation of a data object $\mathbf{o}$ in an iTree τ_i .
- $p(x_u | \tau_i) = \{1, \dots, K\}$ the node path of x_u .
- $|x_u^{(j)} - \eta|$ the deviation of feature j splitted k with threshold η_k .

Intuitively, $g(x_u | \tau_i)$ measures how far the point is from each split boundary *relative to other points*: a small average deviation means the point lies just next to the cut thresholds (in a dense region), whereas a large deviation means the point falls well into one side of the partition (indicating it was in a sparse, outlying region). We further combine the path length $|p(x_u | \tau_i)|$ and the deviation measure:

$$F_{\text{DEAS}}(\mathbf{o} | T) = 2^{-E_{\tau \in T}[|p(x_u | \tau)|]/C(T)} \times E_{\tau \in T}[g(x_u | \tau)],$$

with:

- $C(T)$ a normalization constant i.e. the average path length in an isolation tree for a normal instance.

The first term $2^{-E[|p|]/C(T)}$ is essentially the usual Isolation Forest score (yielding a value in $[0, 1]$ where values near 1 indicate short paths/anomalous). The second term $E[g(x_u | \tau)]$ increases the score for points that have large deviation values (i.e. they tend to lie far from split boundaries, suggesting they reside in sparse regions). Thus, a data point will be deemed highly anomalous by DIF if it is isolated in just a few splits *and* those splits occur in regions where the point is markedly distant from the normal cluster boundaries. In practice, this DEAS scoring has been shown to significantly improve detection accuracy over the standard path-length scoring.

3 Experimental results

3.1 Presentation of the task

3.1.1 Context and objectives

This study is conducted within the AXA Customer Management department, where the use of customer knowledge to prevent contract termination (policy churn) is a core objective since it directly increases acquisition costs and AXA's turnover. Therefore, the operational need is twofold: (1) detect at risk contracts early enough to trigger effective retention actions (e.g., pricing review, coverage adjustment, service outreach) (2) understand the motivation for termination so that interventions are targeted. In this study, we focus on the first operational need that consists in efficient and on time detection. The need to define "efficient" and "timely" remains.

"Timely" entails the need to detect means contract termination early enough to allow prompt action in one's favor, but not so prematurely that interpretation and action levers could become too abstract. For example, on one hand, if it is detected that a client will terminate their contract within the next week, it might not be possible to implement proposals that would dissuade them in time. On the other hand, if we detect that a client will terminate their contract within the next year, the actionable interventions will be imprecise and the full effects of predictions will not be leveraged. **In this case, the department identified that a prediction within three months is the optimal time frame for us to take action and prevent the termination of contracts.**

"Efficient" means that predictions must miss the least possible of terminations, predictions that contract will not be terminated when it will be must be avoided, i.e. we must minimize the false negative in the following predictions. False negatives will be named "FN" in opposition to true negatives "TN"; we will also talk about true positives "TP" in opposition to false positives "FP" (which, respectively, represent the model predicting well a termination and the model predicting a termination when in fact it is not). In order to measure this, **we will use the "recall"**, defined by $\frac{TN}{TN+FN}$. Recall is maximized when false negatives are minimized. Therefore, **we must maximize the recall**. Similarly to false negatives, the minimization of false positives is defined by $\frac{TP}{TP+FP}$ and is called the "precision". A bad precision (i.e. a low precision) has much less impact in our case, but we still have to be cautious about it as it is related with the quantities of TP and TN, which are the main quantities that control the model performance. This said, we define the objective as:

1. The most important goal is to maximize the true positives
2. The sub goal is that when maximizing the true positives, we prefer to minimize false negatives rather than false positives.

In operational reality, data is heterogeneous and temporal; a contract evolves over time with updates to guarantees, premiums, and claims, while customer characteristics and interactions also change. This setting raises three methodological challenges; (1) temporal updates implies multiple versions for each contract, and thus implies strong class imbalance as it generates more class 0 observations (unterminated contracts) than class 1 observations (terminated contracts), (2) mixed data types (numerical and categorical) with potentially nonlinear interactions, and (3) non-stationarity (portfolio, pricing, competition, regulation) that can induce drift.

After introducing our dataset in Section 3.1.2, we will discuss the modeling choices and its motivations in Section 3.1.3.

3.1.2 Data

3.1.2.1 Construction of the dataset We will work with Auto insurance contracts that were subscribed between January 2023 and January 2025 to avoid any drift effect (See Section 2.2.1.2). We built a tabular dataset where each observation is a snapshot of a contract at a given date. The target is a binary label set to 1 if the contract terminates within the three months following the snapshot date, and 0 otherwise. Therefore, for a given contract, we have an observation / a snapshot of its features for every three months between its subscription and its termination (or the end of the period for unterminated contracts, i.e. January 2025). For example, a contract subscribed in January 2024 and terminated in September 2024 will appear 3 times in the dataset, but with the values of its features updated at the corresponding date.

We note explicit biases introduced by some customer features data. Indeed, features such as socio-professional category or marital status are unlikely to be updated at every snapshot (AXA makes a new snapshot every week) during the period under study. This data is only updated when a significant event with the client occurs and the accuracy of the updated values only rely on the client honesty. In order to assess accuracy of these features, we could introduce a confidence coefficient as a new feature but this would require to have access to the date of the last update. As we don't have this information, we can't deal with it. We assume that these features are static features and accept the potential bias this implies.

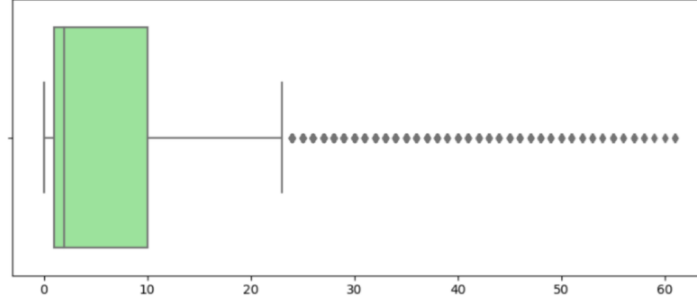
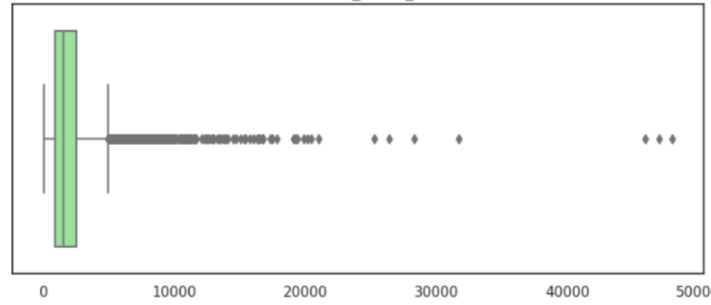
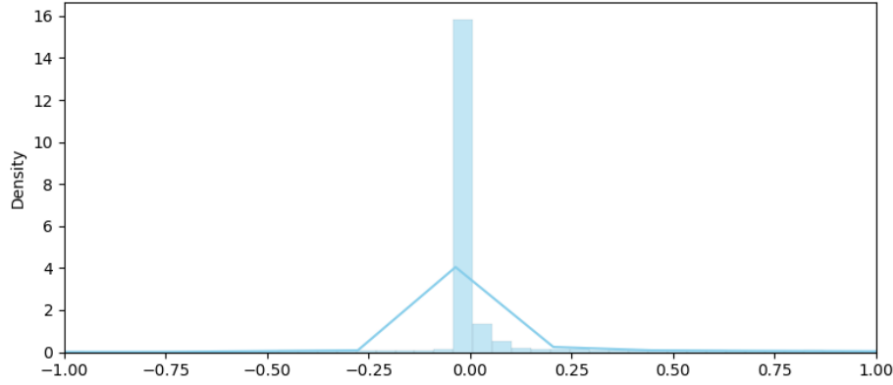
3.1.2.2 Features and data insights The dataset was built based on three types of features, namely contract features, client features, complaints features. The main features and their general characteristics are shown in Table 3 and Table 5. Each of these features is value at the corresponding snapshots of the observations. This is not a exhaustive list because some features are redundant to detail, we only explicit the main features.

Feature	Type	Further information	Null rate (%)
Gender	Categorical		0.0
Country	Categorical		$10e^{-5}$
Socio-professional Category	Categorical		$10e^{-8}$
Family Situation	Categorical	married, single, cohabiting, divorced, widowed	$10e^{-8}$
Entry Product	Categorical	Type of the first product of the client	0.0
Marketing Segment	Categorical	A marketing segmentation indicator	0.0
Contract's Risk Familiy	Categorical	Risk indicator of the contract. For example, 2 wheels or 4 wheels vehicles.	0.0
Product Familiy	Categorical	A product segmentation code	0.0
Management Network	Categorical	The service that manages the contract	$3e^{-4}$
Contract's Region	Categorical	The region where the contract was subscribed	$5e^{-3}$
Client's Age	Numerical		$10e^{-5}$
Client Seniority	Numerical		$2e^{-6}$
Client's Job Seniority	Numerical		0.9
Delay since Client's Last Termination	Numerical	Expressed in months	0.55
Delay since Client's Last Subscription	Numerical	Expressed in months	0
Client's Number of Dependent Children	Numerical		$7e^{-4}$
Client's Salary	Numerical		$9e^{-7}$

Table 3: Feature dictionary with types, and proportion of missing values.

Feature	Type	Further information	Null rate (%)
Client's Significance	Numerical	An indice that indicates how important the client is	0.2
Clients Total Contribution	Numerical	The annual total contribution of the client over all his contracts	0.0
Client's Number of Current Contract	Numerical	The number of contract of the client in all perimeters (life, house, etc.)	0.0
Client's Average Contribution	Numerical	The annual average contribution of the client over all his contracts	0.0
Contract Contribution	Numerical	The annual contribution for the current contract	0.0
Contract Seniority	Numerical		0.0
Contract Contribution Proportion	Numerical	The proportion that the current contract takes in the total contribution of the client	0.0
Contract Contribution at t-3 months	Numerical		0.4
Contract Contribution at initial snapshot	Numerical		8e-7
Contract Contribution Evolution at t-x	Numerical	Evolution of the contract contribution at time t compared to time x.	≈ 0.5

Table 5: Feature dictionary with types and proportion of missing values.

Figure 12: Box plot of **Client's Seniority in months** featureFigure 13: Box plot of **Client's Total Contribution** featureFigure 14: Distribution of **Evolution of Client's Auto Contribution from t-3 to t** in **class 0**

The majority of the features have spread extremes with a relatively tight range from q25 to q75. Figure 12 and Figure 13 illustrate how spread can be the extremes of the features.

Moreover, we observe distributions shifts between the positive class and the negative class in many features. Figures 14 and 15 and Figures 16 and 17 show two by two examples of features which distribution in the positive class deviates from its distribution in the negative class.

At first glance, the distributions in Figures 14 and 15 may seem similar, but it should be noted that we are talking about the percentage evolution of a contribution, meaning that a small delta can translate a large gap. In this way, the distribution in Figure 15, centered around 0.1, contrasts a lot with the distribution in Figure 14, which is centered around -0.1. This supports the intuitive idea that significant changes in contributions can lead to client dissatisfaction and, consequently, contract termination.

Figure 16 suggests that around 50 percents of Auto insurance contracts last at least 9 months (0,75 is visually the seniority that divides equally the total and $0.75 * 12months = 9$) with a left-skewed distribution. The distribution in Figure 17 has a more balanced and centered distribution

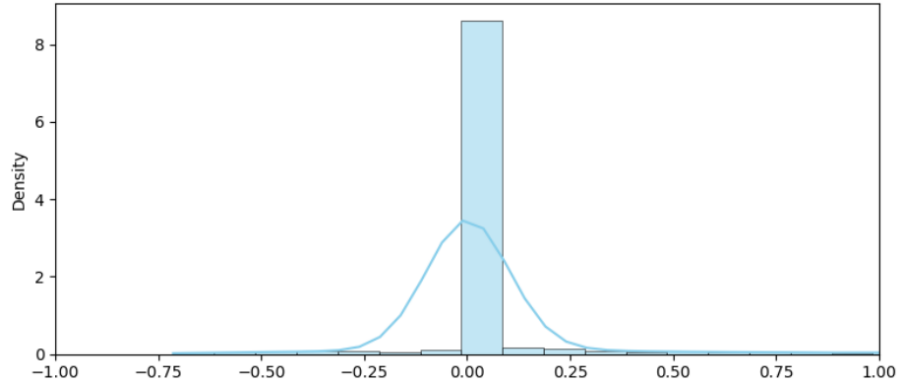


Figure 15: Distribution **Evolution of Client's Auto Contribution from t-3 to t** in **class 1**

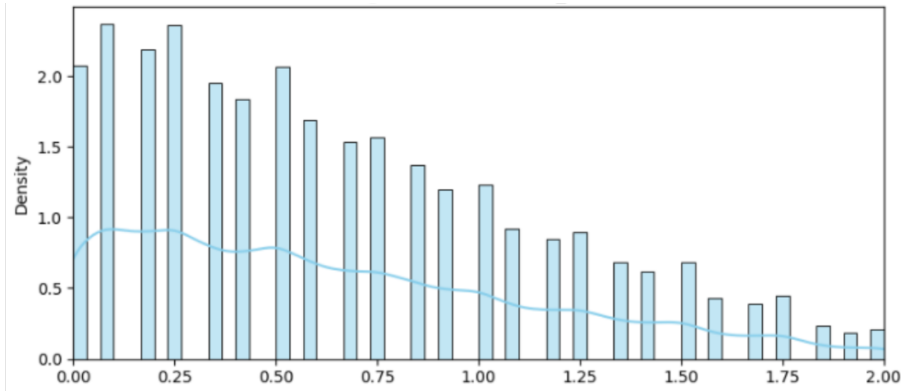
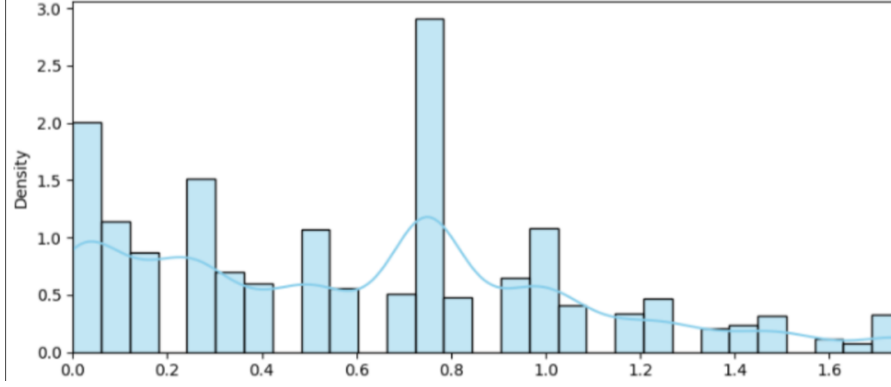


Figure 16: Distribution of **Contract's seniority** in **class 0**

which contrasts with 16. The center of the distribution is complementary with the first one, as it supports the idea that contracts are more likely to be terminated after 9 months.

Overall, the working dataset comprises **6,746,342 samples** with **53 features** (Table 3 and Table 5 do not explicitly state all the features about the clients and contracts contributions and their evolution). The proportion of **the positive class is 2,3 percents** (class 1 = terminated within the next three months).

Figure 17: Distribution **Contract's Seniority** in **class 1**

3.1.3 Experiments settings and modeling choices

In this section, the choices in terms of modeling, data processing, and setting of the experiments are explicitly defined.

3.1.3.1 Data processing: Following the section "Appendix A.1" of [10], we apply the next processings: (1) Categorical variables are one-hot encoded. (2) Continuous variables are standardized only for algorithms that require scale-normalized inputs. (3) In [10], authors apply a mean imputation, but we will use a median imputation instead. Indeed, we can see in Figures 13 and 12 that the features have high extremes and we empirically observed better downstream performance with median imputation.

3.1.3.2 Modeling: The authors of anomaly detection papers sometimes test their models on datasets that are unusual in the context of anomalies (e.g. [10], [12] with marketing campaign dataset) but still match the characteristics of anomalies, such as class imbalance, or expect that class-1 data will significantly deviate from the others. Similarly, we will model this task with a Deep AD model. In fact, our class-1 labels are fully trusted and we have a highly imbalanced dataset, and thus could match the assumptions for AD models. However, class-1 and class-0 data may not differ as much as pure anomalies like in the contexts of cyberattack or outliers detection, and therefore unsupervised modeling might not be efficient as it will surely fail to identify class-1, in addition to not making any profit from the available labeled data. Similarly, semi supervised AD models mostly rely on unlabeled data (e.g. class 0) and won't enable us to take advantage of our labeled data. Indeed, given that the available labels are completely reliable, a weakly-supervised algorithm could be the best option to leverage the poor quantity of class 1. That said, the literature highlights better results with FEAWAD and PreNet, so we make the assumption that these models could be the most suitable for our task. Nevertheless, we will compare them with other deep AD models in the next section. 3.2.1. We will not apply any contamination rate in our study because the labels of unterminated contracts are all 100% reliable, so we do not need to make a model robust to potential hidden anomalies in the normal data.

In order to evaluate what part of the FEAWAD architecture plays a role in the final prediction in imbalanced contexts, we will create ablative implementations. This will allow us to determine how the model improves upon a basic multilayer perceptron (MLP) in this context.

Due to their flexibility, interpretability, and ability to solve heterogeneous tasks, GLMS are a standard in the field of insurance (e.g. [2], [3]). This is why we will use the Logistic Regression, which is a GLM for binary classification, as a baseline for comparison with the anomaly detection models. See its theoretical insights in Section 2.1.3.2.

3.1.3.3 Evaluation: The common metrics to evaluate a binary classifier are the AUC metrics because they give us an average evaluation over all decision thresholds. The state of the art in the field of anomaly detection relies heavily on these metrics [16], [17], [14], [11], [10], [15]. AUC-ROC is the area under the ROC curve, which describes the relationship between true positive rates and false positive rates. This metric evaluates the prediction performance for both the normal class and the abnormal class. In the context of a strong imbalanced dataset, AUC-PR enables us to pay more attention to anomalies. AUC-PR is the area under the precision curve, which describes the relationship between recall and precision. Thus, these metrics can be interpreted as an average score on all possible decision thresholds. If 2,3% is the proportion of the positive class, then any classifier with a AUC-PR superior to 0,023 is better than random and the closer the score to 1, the better the classifier. Any classifier with a AUC-ROC superior to 0,5 is better than random.

In practice, users have to choose a threshold that best fits the 'operational' objectives (e.g., recall, precision). As explained in Section 3.1.1, we have to minimize false negatives, which means maximizing the recall. However, we cannot optimize the threshold only based on the recall, as it would result in choosing 0 being chosen as the threshold. In fact, setting the decision threshold to 0 would classify every observation as 1, giving a recall of 100% and a precision of 0%, but this would not make sense. Actually, we still have to guarantee a minimum precision score. A solution is to choose the threshold based on the 'f1 score', which is a harmonic mean of recall and precision for a given threshold: $f_1 = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$.

In summary, the priority is to evaluate the models based on the **AUC-ROC** and **AUC-PR**; this will be called the global performance. To a leaser extent, the consideration will be in evaluating how well they fit the operational objectives by **comparing their recall on a threshold chosen based on the best f1 score**; this will be called operational performance.

3.1.3.4 Experimentations set up Due to computation resource constraints, we run all experiments on a stratified 200k-samples subset of the data that preserves the global class proportion. Each full experiment (including resampling, processing, training, and evaluation) is repeated five times with different sample seeds, and we report mean $\pm$ standard deviation across runs to account for sampling variability. To ensure a fair comparison, the same splits and processing configuration are applied across models, with no model-specific feature engineering beyond the scale normalization noted above.

The f1-score and their associated recall and precision reported in the studies correspond to the best possible f1-score found on the test prediction after training.

In Appendix, 5.2, the links to original github repositories of the papers and the "deepod" library are given. Self-implementations are given in Appendix 5.3.1 and the code for utils functions is given in 5.3.2. we detail further which code was used for which model. Models and hyper-parameters optimization is performed with the package optuna but we will not detail this part.

3.2 Experiments

In this section, we will first evaluate the main deep anomaly detection models. This will allow us to confirm the modeling choices made in the previous section 3.1.3. Next, we will compare these models to our baselines, i.e. the basic MLP and Logistic Regression models. Deep AD models are primarily based on MLPs. They differ from basic MLPs in terms of their surrogate losses and specific batch sampling methods. To understand which parts of the deep AD model architectures enhance or hinder their performance, we will conduct an ablative study. This will allow us to capture how they behave compared to a basic MLP, particularly in the context of an imbalanced dataset. Finally, we will study how the performance of the models varies according to the proportion of classes.

3.2.1 Benchmark of deep anomaly detection models

Let's first evaluate the performances of deep anomaly detection models previously introduced. We will compare the following models:

1. DevNet: This is the model that inspired the development of FEAAD. We briefly introduced its architecture in 2.2.3.1, but we will not detail it further. We use the code from the **original github** repository of the paper (see section 5.2).
2. FEAAD: We **implemented our own version**. The code is given in in Appendix 5.3.1 and in Appendix 5.3.2.
3. Deep SVDD: Deep SVDD [13] is the model on which Deep SAD was built. We briefly introduced its architecture in Section 2.2.3.3 but its performances are only given for validation purposes, and we will not detail it further. We use the code from the **deepod library** (see section 5.2).
4. Deep SAD: We use the code from the **deepod library** (see section 5.2).
5. PReNet: We use the code from the **original github** repository of the paper (see section 5.2).

The results are given in Table 6. First, they confirm that FEAAD and PReNet perform significantly better. Semi-supervised struggles because they only learn with class-0 data. Unsupervised models struggle even harder because they make too many a priori assumptions for our dataset. In fact, here Deep SVDD is not really better than random. Weakly supervised models manage to leverage the available labeled data and to really improve the performances.

More precisely, DevNet has better recall than FEAAD, but its precision is poorest, affecting its f1 score. Overall, the AUC-PR of FEAAD is better than that of DevNet, which shows that FEAAD has more power in the context of a highly imbalanced dataset.

On his side, PReNet demonstrates the highest performance in AUC metrics and f1-score, marking a big gap with the others. It is far away from the recalls of FEAAD and DevNet but offers a much more balanced compromise, and still obviously favors the recall, which represent the most desirable performances. Its recall and precision scores illustrate the benefits of the pairwise strategy as precision is higher, which means the model predict much less false positives and thus better generalizes to unseen anomalies.

Furthermore, in terms of the improvement between the models, our results are in general identical, if not higher, to those reported in the literature. Specifically, in Table 6 of the PReNet article [12], we can see that on the campaign dataset, PReNet improves AUC-PR by 30% compared to FEAAD, when in our case it improves by 37%. In the FEAAD article [11], FEAAD improves AUC-PR by 4% compared to DevNet, when in our case it improves by 9%.

Models	Type	Global Performance		Operational Performance		
		AUC-ROC	AUC-PR	Best f1-score	Recall	Precision
DevNet	Weakly	0.629±0.014	0.041±0.003	0.053±0.00	0.85 ±0.00	0.028±0.00
FEAWAD	Weakly	0.607±0.016	0.045±0.005	0.060±0.00	0.641 ±0.03	0.03±0.00
Deep SVDD	Unsupervised	0.534±0.004	0.028±0.001	0.051±0.01	0.68±0.03	0.03±0.01
Deep SAD	Semi	0.561±0.014	0.038±0.007	0.07±0.01	0.16±0.04	0.04±0.01
Deep iForest	Unsupervised	0.556±0.032	0.031±0.004	0.07±0.01	0.46±0.05	0.05±0.04
PReNet	Weakly	0.651 ±0.005	0.062 ±0.007	0.109 ±0.01	0.151±0.04	0.109 ±0.05

Table 6: Performances of deep anomaly detection models (mean $\pm$ standard deviation) on 5 runs. Global performance metrics enable to measure how well the models behave over all thresholds. Operational Performance enable to measure the best f1-score, and its associated recall and precision.

3.2.2 Anomaly detection-based models vs basic classification models

3.2.2.1 Ablative implementation What mostly distinguishes FEAWAD and PReNet from a basic MLP is the use of representation features, a surrogate loss, or a specific training process with custom batch sampling functions (see details in Section 2.2.3.1, 2.2.3.2). FEAWAD also takes advantage of a pre-trained autoencoder, but this part is not so specific to anomaly detection architecture. In fact, pre-trained autoencoders are often used in insurance and other fields to enhance the performance of a neural network [18]. We will implement the following models:

1. **MLP:** is a basic perceptron, and the weights of its first layers are initialized with the encoding weights of a pre-trained autoencoder. Its output activation function is the sigmoid function with a binary cross-entropy loss. Its architecture is depicted in Figure 18 where we can notice that the encoded features are the same than the variable h in FEAWAD architecture in Figure 19. The corresponding functions in Appendix 5.3.1 are *auto_encoder()* and *pretrained_classifier()*.
2. **AD_MLP:** is like the MLP model, but it concatenates the input of the classifier with h , e , r features, which are variables of FEAWAD model detailed in Section 2.2.3.1. Moreover, its autoencoder is pre-trained with only class 0 samples just like in FEAWAD. This approach is analogous to the use of AD semi-supervised models, which facilitate the acquisition of a priori knowledge on normal data. Its output activation function is also the sigmoid function with a binary cross-entropy loss. Its architecture is depicted in Figure 19. The corresponding functions in Appendix 5.3.1 are *auto_encoder()* and *pretrained_anomaly_clf()*.
3. **FEAWAD:** What differs between the FEAWAD and the AD_MLP models is (1) the max-margin-like loss function (2) the training process with a specific batch sampling. During the end-to-end training, the batch is made up of half anomalies and half normal data. Everything is detailed in Section 2.2.3.1. We used the code of the original repository of the authors, given in Appendix 5.2. However, they used an old version of Tensorflow so we upgraded it. The code is given in in Appendix 5.3.1 and in Appendix 5.3.2.
4. **PReNet** is the same as in previous Section 6.
5. **Logistic Regression** is our insurance baseline for comparison and we used the scikit-learn package.

3.2.2.2 Performances The models performances are reported in Table 7. We can see that AD_MLP performs better than the other models in most metrics. First, it highlights how the features h , e and r enable the classifier to globally improve its predictions with an improvement of $\approx 2\%$ in

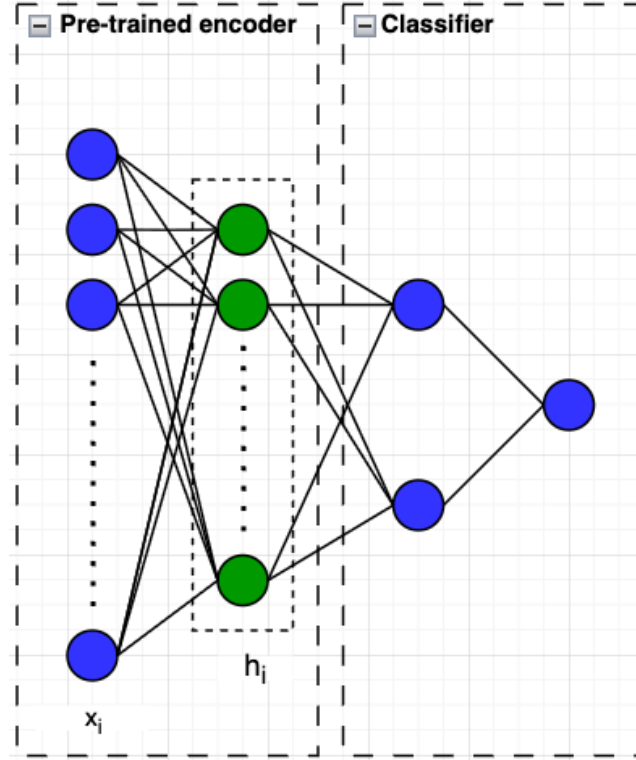


Figure 18: Architecture of the basic MLP model with a pre-trained encoder. The number of layers is for illustration purposes.

AUC-ROC and $\approx 6,8\%$ in AUC-PR. If we reformulate the difference of the architectures of MLP and AD_MLP, we could say that MLP feeds a classifier with the variable h , and AD_MLP feeds the classifier with h , e , r . In Table III of the FEAAD paper [11], the authors also carried out an ablative study on which they measured an improvement of $\approx 2,7\%$ in AUC-ROC and $\approx 1,5\%$ in AUC-PR when adding e and r to h . If we look at the type of prediction that it improved, we observe that the recall was enhanced (in opposite to precision).

When adding the max-margin-like loss and the batch sampling technique, i.e. moving from AD_MLP to FEAAD, we can observe a significant drop in the global performances. This is due to a large imbalance in the model’s capabilities because it has a very high recall, which has a very negative impact on precision.

Logistic regression has moderate global performance and is slightly below the basic MLP in terms of most metrics. However, it demonstrated the best overall precision and by consequence, the worst

Models	Global Performance		Operational Performance		
	AUC-ROC	AUC-PR	Best f1-score	Recall	Precision
MLP	0.643 $\pm$ 0.008	0.073 $\pm$ 0.009	0.114 $\pm$ 0.02	0.105 $\pm$ 0.00	0.247 $\pm$ 0.11
AD_MLP	0.656 $\pm$ 0.006	0.078 $\pm$ 0.011	0.120 $\pm$ 0.02	0.125 $\pm$ 0.01	0.196 $\pm$ 0.10
FEAWAD	0.607 $\pm$ 0.016	0.045 $\pm$ 0.005	0.060 $\pm$ 0.00	0.641 $\pm$ 0.03	0.03 $\pm$ 0.00
PRNet	0.651 $\pm$ 0.005	0.062 $\pm$ 0.007	0.109 $\pm$ 0.01	0.151 $\pm$ 0.04	0.109 $\pm$ 0.05
Log. Reg.	0.634 $\pm$ 0.009	0.069 $\pm$ 0.009	0.112 $\pm$ 0.01	0.096 $\pm$ 0.04	0.267 $\pm$ 0.10

Table 7: Performances of each model (mean $\pm$ standard deviation) on 5 runs. Global performance metrics enable to measure how well the models behave over all thresholds. Operational Performance enable to measure the best best f1-score, and its associated recall and precision.

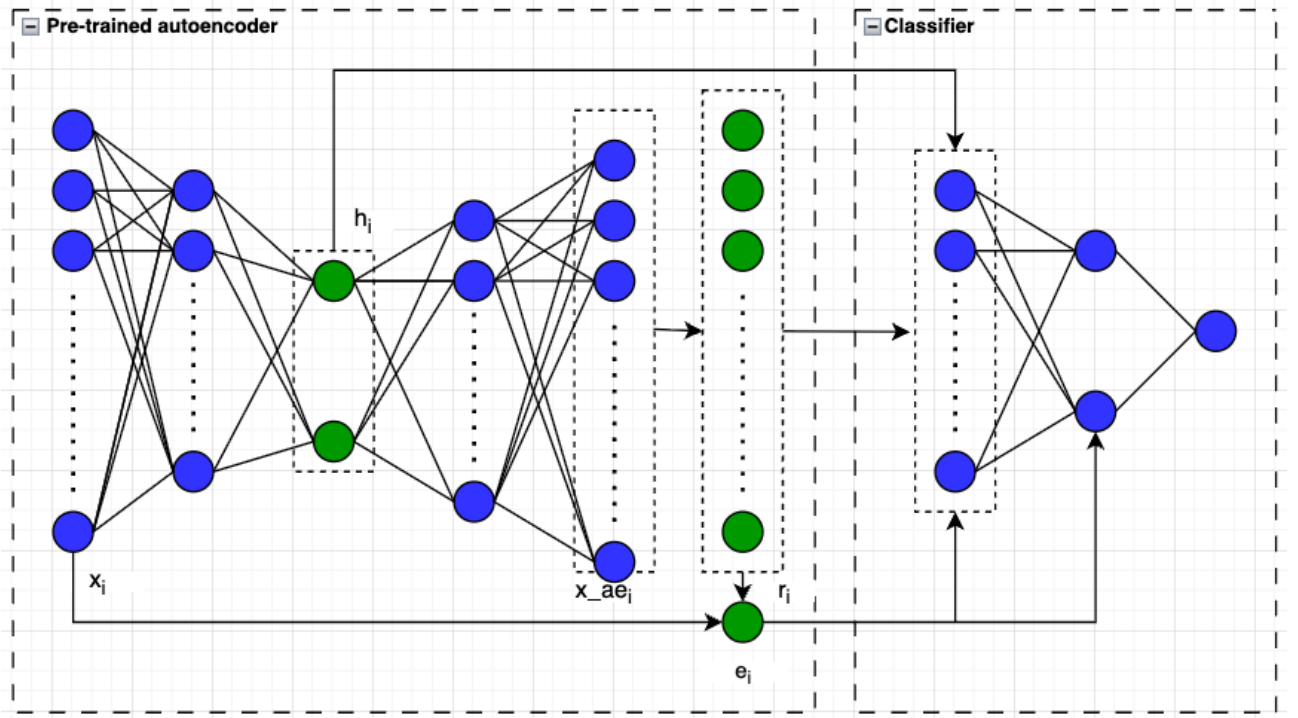


Figure 19: Architecture of AD_MLP with representations features h , e , r from FEAWAD model. The number of layers is for illustration purposes.

recall.

PReNet demonstrates pretty decent performance, with a slightly lower AUC-PR than Logistic Regression. It is interesting to see in Figure 20 that the precision curve gives a "symmetrical" threshold in terms of the f1 score and the associated recall and precision compared to that of Logistic Regression. This highlights the ability of its surrogate dataset and label strategy to reduce false positives and outperform the FEAWAD max-margin loss and sample strategy.

In summary, the results demonstrate that each of the features added by the anomaly detection models improves recall. Specifically, the results confirm their ability to identify class-1 samples and therefore minimize false negatives. In contrast, classical models identify fewer class-1 samples, but minimize false positives. In AD_MLP, the representation variables h , e and r enable a strong global improvement and establish the best performances. Loss and batch sampling strongly improve recall, but deteriorate the other metrics a lot. For illustration purposes, we give examples of precision curves (Figure 20) of Logistic Regression, AD_MLP, and PReNet. We can see how AD_MLP outperforms other models in terms of AUC-PR, giving a good middle point between Recall and Precision. PReNet curve stays slightly above other curves on the right side, showing its ability to maintain better precision than others when increasing the recall (i.e. increasing the decision threshold).

In the next section, we are going to study how the models behave when the proportion of positive class varies.

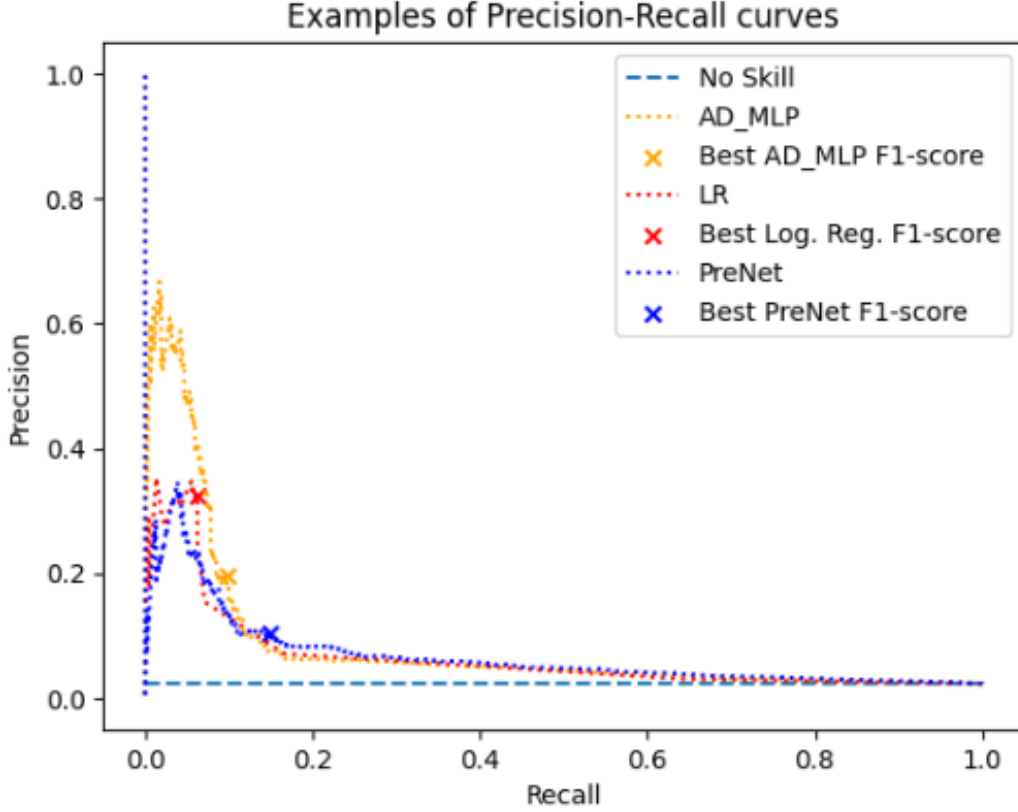


Figure 20: Example of precision curves of the models on a test dataset of $\approx 20,000$ samples.

3.2.3 The impact of variation in class proportions

In the literature on anomaly detection, and especially in the weakly supervised context [11], [10], one key question is to know how resilient are the models to variation in the number of labeled anomalies. These models are designed to make the most of the small quantity of labels available. To further understand the benefits of the anomaly detection features, we studied the behaviors of our models with different level of class-1 proportion.

First, we built a dataset with a proportion of 1% of class 1 by sub-sampling the main dataset. Then we managed to create two datasets with proportions of 10% and 30% by increasing the length of the studied period and / or decreasing the number of snapshots for each unterminated contract. Figures 21, 22 and 23 illustrate our results.

First, we can see that AD_MLP has the best AUC-ROC, AUC-PR, and F1 scores for all levels of class 1 proportion. Interestingly, the closer the proportion of class 1 is to the higher end of the range, the closer AD_MLP is to the MLP scores. In contrast, the smaller the proportion of class 1, the greater the difference in performance between the two models. In addition to Section 3.2.2, this confirms the benefits that the prediction takes from h, e, r and the specific batch sampling for pre-training. Moreover, it highlights that these features have a strong capacity of identifying anomalies in a very low class 1 proportion context ($< 3\%$).

More specifically the trend of the recalls reverses between anomaly detection models (AD_MLP and FEAWAD) and classical models (MLP, Logistic Regression). We see that the recall of MLP becomes slightly better than the recall of AD_MLP with a class 1 proportion of 30% but otherwise is lower, highlighting the power of h, e, r and specific batch sampling in pre-training. However, the when the class-1 proportion increase we observe that the precision of AD_MLP becomes better than MLP and Logistic Regression. Logistic Regression shows the biggest gap as it goes from worst recall score when class 1 is at 1% (meaning it has not better skills than random) to best recall score when class 1 is at

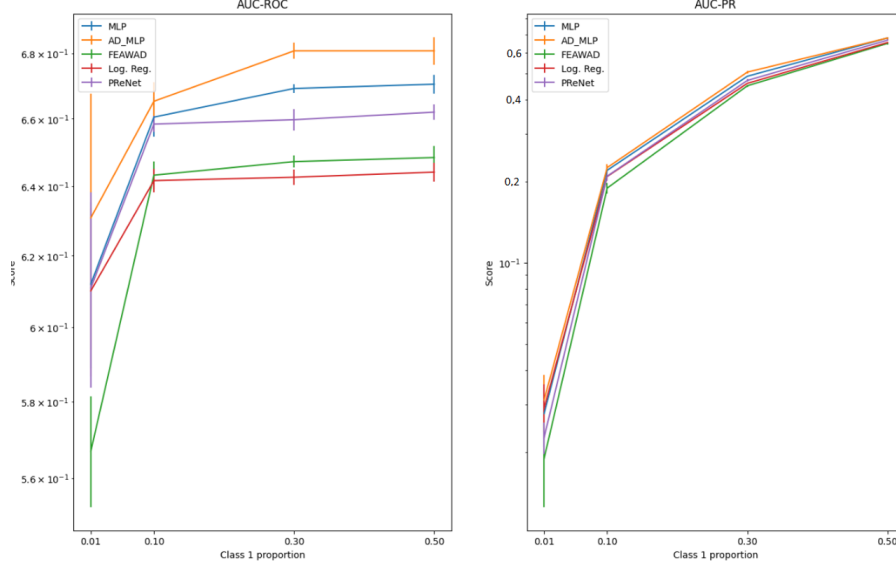


Figure 21: Global performances of the models with variation in the class proportions. We use a log scale for Y axis.

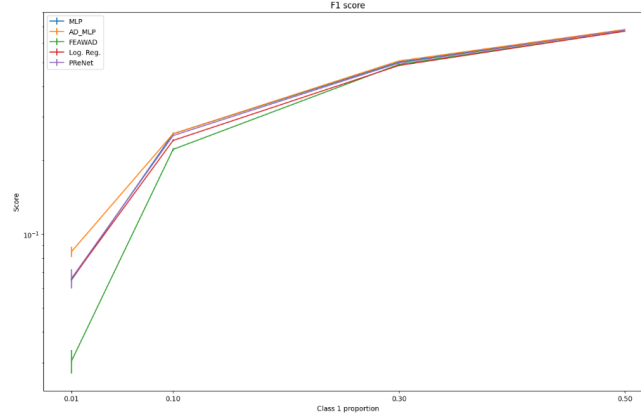


Figure 22: F1 score of the models with variation in the class proportions. We use a log scale for Y axis.

30%.

FEAWAD has the worst score in almost every metrics, but confirms the capacity of its architecture to ensure a high recall whether the class 1 proportion is high or low.

PReNet does not have very high global performances but is still better than Logistic Regression and FEAWAD in the overall. However, PReNet demonstrates a strong capacity to keep the precision high when the number of labels is very low. Its recall is lower than that of AD_MLP but not so much, and it is still better than that of MLP, enabling it to have the second best f1 score for all levels of class proportion. Moreover, its recall becomes better than that of MLP when the number of labels increases.

This being said, we can now confirm that the features introduced by FEAWAD (h, e, r, batch sampling and max-margin-like loss) enable to leverage crucial information about the data and guarantee to maximize the recall when the proportion of class 1 is reasonably low but lose of their interest when this proportion increases to more usual quantities. We also confirm that the max-margin loss favors too much the recall and thus really deteriorates overall performance in all levels of class proportion.

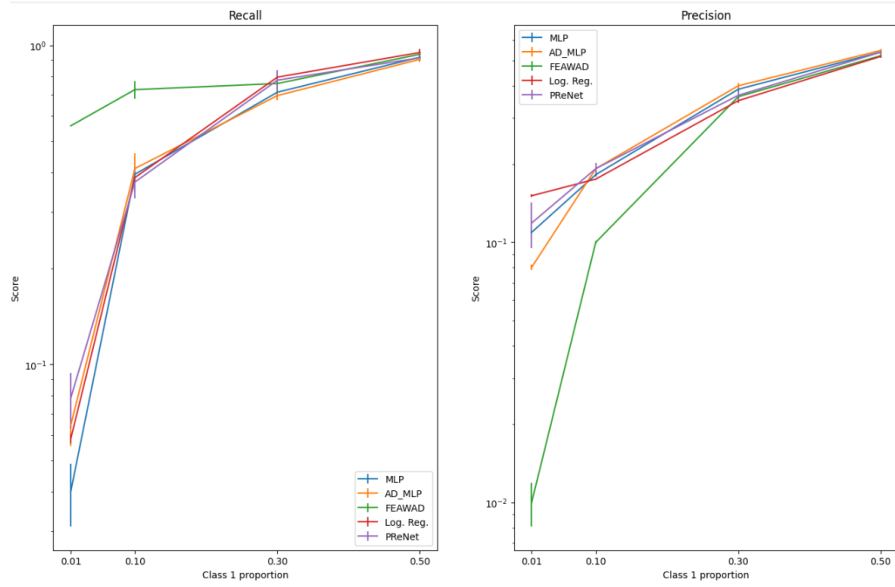


Figure 23: Operational performances-Recall and Precision-of the models with variation in the class proportions. We use a log scale for Y axis.

To mitigate this issue, our results suggest that using only the h , e , r variables and the pre-training specific batch sampling, as in AD_MLP, achieves the best results so far.

4 Conclusion

In this Master’s thesis, we approached the task of predicting contract termination as an anomaly detection problem.

4.1 Carried out work

We compared the state-of-the-art models and got the same results, in terms of improvement from one model to another, as in the literature, confirming a possible use of these models for our dataset. This benchmark demonstrated that weakly supervised models outperform unsupervised and semi supervised models in the context of our task, i.e. the prediction of churn with client knowledge-based features.

We selected FEAAD and PReNet, two weakly supervised models that have shown to be the most adapted to our task. To measure their performances, we compared them with the standard in the insurance field: logistic regression, a binary classifier from the Generalized Linear Models family. To understand what part of their architectures helps in an imbalanced dataset context, we conducted an ablative study. More precisely, we demonstrated that, while each of the features introduced by FEAAD improves recall, they also negatively impact precision, resulting in a deterioration of the AUC-PR. Thanks to the AD_MLP model, we identified that the specific batch sampling of only normal data in autoencoder pre-training, and the h, e and r representation variables together gave the best global and operational performances in our study compared to other models. Moreover, it only slightly deteriorate precision in low class-1 proportion context, quickly returning back to the best precision when the number of labels increases.

Finally, we studied the impact of the variation in class-1 proportion and demonstrated that the anomaly detection-based models AD_MLP perform better than others but exploits its full potential when the class-1 proportion is unusually low.

4.2 Limitations:

If we take a step back, we see that the results are still pretty poor. This could be the consequence of several things.

Anomaly detection studies assume that the data are perfectly clean. Indeed, an anomaly is often close to an outlier and dirty data can be interpreted as an anomaly when, in fact, is a normal data but with bias. Specifically, weakly-supervision makes the assumption that the labeled data is totally clean and that the unlabeled data contains unknown anomalies. These unknown anomalies could be dirty normal data in our case and thus prevent the model of accurately learning on these. A limitation in our data is the potential bias introduced by client-provided information, which we considered negligible. In practice, some of our client records inevitably contain noise or inaccuracies. This stems from the fact that a portion of the data is self-reported or indirectly collected. For example, through “snapshots” of contracts and supporting documents provided by clients. As a result, the reliability of certain features depends on how consistently and precisely the client communicates the required details. This known bias must be kept in mind, since it can affect both model training and evaluation by introducing irregularities that are not true anomalies but artifacts of imperfect data collection.

Furthermore, the datalab used for this study is recent and a number of issues pertaining to data quality remain to be addressed. Despite the fact that a considerable amount of cleaning has already been carried out, it is believed that there are still non-negligible inaccuracies.

4.3 Possible future works:

Compared to classical machine learning, deep learning enables models to have a higher capacity of features representation. However, client data are both numerical and categorical, whereas most

existing methods were built for numerical data. Therefore, an upgrade could aim to improve the architecture of a deep anomaly detection model so it handles better mixed data. For example, L. Delong and A. Kozak [18] proposed an autoencoder for mixed data in the context of prediction of the number of claims.

Finally, we made the assumption that we are working with point anomalies, but we could either look at the problem as a group anomaly detection task. Indeed, all the snapshots of a given terminated contract can be interpreted as a group of samples that, except the last snapshot, can individually be normal data (i.e. class-0 / unterminated contract) but all together represent an anomaly (i.e. class-1 / a terminated contract). To address this type of problem, we can find many papers in the literature. In the last few years, the transformer architecture with the mechanism of attention has pushed the state-of-the-art performances with breaking results. Group anomaly detection field is no exception [19], [20], [21]. One really interesting work would be to apply these models to our task.

References

- [1] J. A. Nelder, R. W. M. Wedderburn, *Generalized Linear Models*, 1972.
- [2] Casualty Actuarial Society, *Generalized Linear Models for insurance rating*, 2025.
- [3] S. Kačková, L. Křivánková, *Generalized Linear Models in vehicle insurance*, 2014.
- [4] Z. Rouaine, A. Amrani, *Prediction of the act of termination of car insurance contract at the end of their term*, 2023.
- [5] P. Fraoughi, S. Li, J. Ren, *The Applications of Generalized Poisson Regression Models to Insurance Claim Data*, 2023.
- [6] Yangha Chun, *Machine Learning-Based Termination Prediction Modeling - University Yongin* 2024.
- [7] G. Pang, C. Shen, L. Cao, A. Van Den Hengel *Deep Learning for Anomaly Detection: A Review*, 2021.
- [8] F. T. Liu, K. M. Ting, Z. H. Zhou *Isolation Forest*, 2008.
- [9] A. Dairi, F. Harrou, Y. Sun *Efficient Driver Drunk Detection by Sensors: A Manifold Learning-Based Anomaly Detector*, 2022.
- [10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun, *Deep Residual Learning for Image Recognition*, 2019.
- [11] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson, *How transferable are features in deep neural networks?* , 2021.
- [12] G. Pang, C. Shen, H. Jin, A. Hengel *Deep Weakly-supervised Anomaly Detection*, 2023.
- [13] L. Ruff, R. A. Vandermeulen, N. Görnitz, L. Deecke, E. Müller, S. A. Siddiqui, A. Binder, E. Müller, M. Kloft. *Deep One-Class Classification*, 2018.
- [14] L. Ruff, R. A. Vandermeulen, N. Görnitz, A. Binder, E. Müller, K. R. Müller, M. Kloft. *Deep Semi-supervised Anomaly Detection*, 2020.
- [15] H. Xu, G. Pang, Y. Wang, Y. Wang *Deep Isolation Forest for Anomaly Detection*, 2023
- [16] G. O. Campos, A. Zimek, J. Sander, R. J. Campello, B. Micenkova , E. Schubert, I. Assent, and M. E. Houle, *On the evaluation of unsupervised outlier detection: measures, datasets, and an empirical study - Data Mining and Knowledge Discovery*, vol. 30, no. 4, pp. 891–927, 2016
- [17] C. D. Manning, H. Schütze, and P. Raghavan, *Introduction to information retrieval*, Cambridge university press, 2008
- [18] L. DeLong, A. Kozak, *The use of autoencoders for training neural networks with mixed categorical and numerical features*, 2023
- [19] A. Lohrer, D. Malik, C. Zelenka, P. Kroger, *GADformer: A Transparent Transformer Model for Group Anomaly Detection on Trajectories*, 2024
- [20] R. Chalapathy, E. Toth, S. Chawla, *Group Anomaly Detection using Deep Generative Models*, 2018
- [21] H. Gan, H. Zheng, J. Nescolarde Selva, Y. Chen, Z. Wu, F. Ullah, *TCAD-Net: Integrating Transformer and Convolution for Weakly Supervised Anomaly Data Discovery*, 2025

5 Appendix

5.1 Requirements

We implemented this project under:

- pytorch==2.8.0
- torch-cluster==1.5.9
- numpy==1.24
- pandas==1.5.3
- tensorflow==2.16.1
- scikit-learn==1.2

5.2 Anomaly Detection existing code

The models are directly available in the original github repositories of the papers.

- DevNet: <https://github.com/GuansongPang/deviation-network>
- FEAAD: https://github.com/yj-zhou/Feature_Encoding_with_AutoEncoders_for_Weakly-supervised_Anomaly_Detection
- Deep SAD: <https://github.com/lukasruff/Deep-SAD-PyTorch>
- PreNet: <https://github.com/mala-lab/PReNet>

The models are also available in the deepod library: <https://github.com/xuhongzuo/DeepOD>, which currently has 540 stars.

5.3 Implementations

5.3.1 Models

We implemented FEAAD, MLP, and AD_MLP:

```

1  def auto_encoder(
2      input_shape,
3      learning_rate=0.0005,
4      intermediate_dim=128,
5      enc_dim=64,
6      act_fct="relu",
7  ):
8      x_input = Input(shape=input_shape)
9      input_dim = x_input.shape[1]
10
11     input_vector = Dense(input_dim, kernel_initializer='glorot_normal', use_bias=True, activation=act_fct, name =
12     ↪ 'ain')(x_input)
13     en1 = Dense(intermediate_dim, kernel_initializer='glorot_normal', use_bias=True, activation=act_fct, name =
14     ↪ 'ae1')(input_vector)
15     en2 = Dense(enc_dim, kernel_initializer='glorot_normal', use_bias=True, activation=act_fct, name = 'ae2')(en1)
16     de1 = Dense(intermediate_dim, kernel_initializer='glorot_normal', use_bias=True, activation=act_fct, name =
17     ↪ 'ad1')(en2)
18     de2 = Dense(input_dim, kernel_initializer='glorot_normal', use_bias=True, activation=act_fct, name =
19     ↪ 'ad2')(de1)
20
21     encoder = Model(input_vector, en2, name="encoder")
22     autoencoder = Model(x_input, de2, name="autoencoder")
23     adm = Adam(learning_rate=learning_rate)
24     autoencoder.compile(loss=mean_squared_error, optimizer=adm)
25     return encoder, autoencoder

```

```

23 def pretrained_classifier(
24     encoder,
25     autoencoder=None,
26     learning_rate=0.0001,
27     hidden_dim1=64,
28     hidden_dim2=32,
29     act_fct="relu",
30 ):
31     x_input = Input(shape=encoder.input_shape[1:])
32     input_dim = x_input.shape[1]
33
34     encoded_feature = encoder(x_input)
35     mlp_layer1 = Dense(hidden_dim1, kernel_initializer='glorot_normal', use_bias=True, activation=act_fct, name =
36     ↪ 'mlp1')(encoded_feature)
37     mlp_layer2 = Dense(hidden_dim2, kernel_initializer='glorot_normal', use_bias=True, activation=act_fct, name =
38     ↪ 'mlp2')(mlp_layer1)
39     output = Dense(1, kernel_initializer='glorot_normal', use_bias=True, activation='sigmoid', name =
40     ↪ 'score')(mlp_layer2)
41
42     clf = Model(x_input, output)
43     adm = Adam(learning_rate=learning_rate)
44     clf.compile(
45         loss=binary_crossentropy,
46         optimizer=adm,
47         metrics=[tf.keras.metrics.AUC(name='auc_roc', curve="ROC"), tf.keras.metrics.AUC(name='auc_pr',
48         ↪ curve="PR")]
49     )
50     return clf
51
52 def pretrained_anomaly_clf(
53     encoder,
54     autoencoder,
55     learning_rate=0.0001,
56     hidden_dim1=64,
57     hidden_dim2=32,
58     act_fct="relu",
59 ):
60     """classifier with features representations values h, r, e"""
61     x_input = Input(shape=autoencoder.input_shape[1:])
62     input_dim = x_input.shape[1]
63
64     # Create MLP input
65     autoencoder_output = autoencoder(x_input)
66     sub = Subtract()([x_input, autoencoder_output]) # Autoencoder's input/output diff
67     h = encoder(x_input) # Latent features
68     e = Lambda(lambda x: tf.norm(x, ord = 2, axis=1))(sub) # Reconstruction error
69     e = Reshape((1,))(e)
70     r = Lambda(lambda x: tf.truediv(x[0], x[1]))([sub, e]) # Reconstruction residual vector
71     mlp_input = concatenate([h, e, r], axis=1)
72
73     # MLP (Dense + e)
74     mlp_layer1 = Dense(hidden_dim1, kernel_initializer='glorot_normal', use_bias=True, activation=act_fct, name =
75     ↪ 'mlp1')(mlp_input)
76     mlp_layer1 = concatenate([mlp_layer1, e], axis=1)
77     mlp_layer2 = Dense(hidden_dim2, kernel_initializer='glorot_normal', use_bias=True, activation=act_fct, name =
78     ↪ 'mlp2')(mlp_layer1)
79     mlp_layer2 = concatenate([mlp_layer2, e], axis=1)
80     output = Dense(1, kernel_initializer='glorot_normal', use_bias=True, activation='sigmoid', name =
81     ↪ 'output')(mlp_layer2)
82
83     clf = Model(x_input, output)
84     adm = Adam(learning_rate=learning_rate)
85     clf.compile(
86         loss=binary_crossentropy,
87         optimizer=adm,
88         metrics=[tf.keras.metrics.AUC(name='auc_roc', curve="ROC"), tf.keras.metrics.AUC(name='auc_pr',
89         ↪ curve="PR"), tf.keras.metrics.F1Score(name='f1_score', threshold=0.5)]
90     )
91     return clf
92
93 def feawad(
94     encoder,
95     autoencoder,

```

```

88     learning_rate=0.0001,
89     hidden_dim1=256,
90     hidden_dim2=32,
91     act_fct="swish",
92 ):
93     """classifier with features representations values h, r, e"""
94     x_input = Input(shape=autoencoder.input_shape[1:])
95     input_dim = x_input.shape[1]
96
97     # Create MLP input
98     autoencoder_output = autoencoder(x_input)
99     sub = Subtract()([x_input, autoencoder_output]) # Autoencoder's input/output diff
100    h = encoder(x_input) # Latent features
101    e = Lambda(lambda x: tf.norm(x,ord = 2,axis=1))(sub) # Reconstruction error
102    e = Reshape((1,))(e)
103    r = Lambda(lambda x: tf.truediv(x[0], x[1]))([sub, e]) # Reconstruction residual vector
104    mlp_input = concatenate([h, e, r], axis=1)
105
106    # MLP (Dense + e)
107    mlp_layer1 = Dense(hidden_dim1, kernel_initializer='glorot_normal',use_bias=True,activation=act_fct,name =
    ↪ 'mlp1')(mlp_input)
108    mlp_layer1 = concatenate([mlp_layer1, e],axis=1)
109    mlp_layer2 = Dense(hidden_dim2, kernel_initializer='glorot_normal',use_bias=True,activation=act_fct,name =
    ↪ 'mlp2')(mlp_layer1)
110    mlp_layer2 = concatenate([mlp_layer2, e],axis=1)
111    output = Dense(1, kernel_initializer='glorot_normal',use_bias=True, activation='linear', name =
    ↪ 'output')(mlp_layer2)
112    combined_output = concatenate([output, e], axis=1, name="combined_output")
113
114    def multi_loss(y_true, combined_output):
115        alpha = 5
116        lbda = 1
117
118        dev = tf.squeeze(combined_output[:, 0:1], axis=1) # score output
119        e = tf.squeeze(combined_output[:, 1:2], axis=1) # norm of reconstruction residual
120
121        inlier_loss = tf.abs(dev)
122        outlier_loss = tf.abs(tf.maximum(alpha - dev, 0.0))
123        outlier_sub_loss = tf.abs(tf.maximum(alpha - e, 0.0))
124
125        loss = (1.0 - y_true) * (inlier_loss + e) + y_true * (outlier_loss + outlier_sub_loss)
126        # loss = (1.0 - y_true) * inlier_loss + y_true * outlier_loss + lbda * ((1.0 - y_true) * e + y_true *
    ↪ outlier_sub_loss)
127        return loss
128
129    clf = Model(x_input, combined_output)
130    adm = Adam(learning_rate=learning_rate)
131    clf.compile(
132        loss=multi_loss,
133        optimizer=adm,
134    )
135    return clf

```

5.3.2 Utils

```

1  def one_run(
2      X_train,
3      X_val,
4      y_train,
5      y_val,
6      input_shape,
7      ae_model_fct,
8      ae_params,
9      clf_model_fct,
10     clf_params,
11     patiences={"ae": 10, "clf": 10},
12     verbose=1
13 ):
14     print("Pre-training encoder")

```

```

15     encoder, autoencoder = ae_model_fct(input_shape=input_shape,
16                                         **ae_params["model_params"])
17     ae_hist = autoencoder.fit(x=X_train,
18                             y=X_train,
19                             validation_data=(X_val, X_val),
20                             callbacks=[tf.keras.callbacks.EarlyStopping(monitor='val_loss',
21                                 patience=patiences["ae"],
22                                 restore_best_weights=True)],
23                             verbose=verbose,
24                             **ae_params["train_params"])
25
26     print("Training classifier")
27     clf = clf_model_fct(encoder=encoder,
28                         autoencoder=autoencoder,
29                         **clf_params["model_params"])
30     clf_hist = clf.fit(x=X_train,
31                      y=y_train,
32                      validation_data=(X_val, y_val),
33                      callbacks=[tf.keras.callbacks.EarlyStopping(monitor='val_loss',
34                          patience=patiences["clf"],
35                          restore_best_weights=True)],
36                      verbose=verbose,
37                      **clf_params["train_params"])
38
39     return clf, encoder, clf_hist, ae_hist
40
41
42     rng = np.random.RandomState(42)
43     MAX_INT = np.iinfo(np.int32).max
44
45     def auto_encoder_batch_generator_sup(x, inlier_indices, batch_size, nb_batch, rng):
46         """auto encoder batch generator"""
47
48         rng = np.random.RandomState(rng.randint(MAX_INT, size = 1))
49         counter = 0
50         while 1:
51             ref, training_labels = AE_input_batch_generation_sup(x, inlier_indices, batch_size, rng)
52             counter += 1
53             yield(ref, training_labels)
54             if (counter > nb_batch):
55                 counter = 0
56
57     def AE_input_batch_generation_sup(train_x, inlier_indices, batch_size, rng):
58         '''
59             batches of samples. This is for csv data.
60             Alternates between positive and negative pairs.
61         '''
62         dim = train_x.shape[1]
63         ref = np.empty((batch_size, dim))
64         training_labels = np.empty((batch_size, dim))
65         n_inliers = len(inlier_indices)
66         for i in range(batch_size):
67             sid = rng.choice(n_inliers, 1)
68             ref[i] = train_x[inlier_indices[sid]]
69             training_labels[i] = train_x[inlier_indices[sid]]
70         return np.array(ref), np.array(training_labels)
71
72     def batch_generator_sup(x, outlier_indices, inlier_indices, batch_size, nb_batch, rng):
73         """batch generator"""
74
75         rng = np.random.RandomState(rng.randint(MAX_INT, size = 1))
76         counter = 0
77         while 1:
78             ref, training_labels = input_batch_generation_sup(x, outlier_indices, inlier_indices, batch_size, rng)
79             ref = ref.astype(np.float32)
80             training_labels = training_labels.astype(np.float32)
81
82             counter += 1
83             yield(ref, training_labels)
84             if (counter > nb_batch):
85                 counter = 0
86
87     def input_batch_generation_sup(train_x, outlier_indices, inlier_indices, batch_size, rng):
88         '''
89             batches of samples. This is for csv data.

```

```

88     Alternates between positive and negative pairs.
89     '''
90     dim = train_x.shape[1]
91     ref = np.empty((batch_size, dim))
92     training_labels = []
93     n_inliers = len(inlier_indices)
94     n_outliers = len(outlier_indices)
95     for i in range(batch_size):
96         if (i % 2 == 0):
97             sid = rng.choice(n_inliers, 1)
98             ref[i] = train_x[inlier_indices[sid]]
99             training_labels += [0]
100         else:
101             sid = rng.choice(n_outliers, 1)
102             ref[i] = train_x[outlier_indices[sid]]
103             training_labels += [1]
104     return np.array(ref), np.array(training_labels)
105
106
107 def run_feawad(
108     X_train,
109     X_val,
110     y_train,
111     y_val,
112     input_shape,
113     ae_model_fct,
114     ae_params,
115     clf_params,
116     patiences={"ae": 7, "clf": 7},
117     verbose=1
118 ):
119     print("Preprocess data")
120     X_train, X_val = X_train.to_numpy(), X_val.to_numpy()
121     outlier_indices = np.where(y_train == 1)[0]
122     inlier_indices = np.where(y_train == 0)[0]
123
124     print("Pre-training encoder")
125     encoder, autoencoder = ae_model_fct(input_shape=input_shape,
126                                         **ae_params["model_params"])
127     ae_hist = autoencoder.fit(auto_encoder_batch_generator_sup(X_train, inlier_indices, batch_size=512,
128     ↪ nb_batch=20, rng=rng),
129                             validation_data=(X_val, X_val),
130                             callbacks=[tf.keras.callbacks.EarlyStopping(monitor='val_loss',
131                                 patience=patiences["ae"],
132                                 restore_best_weights=True)],
133                             verbose=verbose,
134                             **ae_params["train_params"])
135
136     print("Training classifier")
137     clf = feawad(encoder=encoder,
138                 autoencoder=autoencoder,
139                 **clf_params["model_params"])
140     clf_hist = clf.fit(
141         batch_generator_sup(X_train, outlier_indices, inlier_indices,
142     ↪ batch_size=clf_params["train_params"]["batch_size"], nb_batch=20, rng=rng),
143         epochs=clf_params["train_params"]["epochs"],
144         steps_per_epoch = 20,
145         validation_data=(X_val, y_val),
146         callbacks=[tf.keras.callbacks.EarlyStopping(monitor='val_loss',
147             patience=patiences["clf"],
148             restore_best_weights=True)],
149         verbose=verbose)
150
151     return clf, encoder, clf_hist, ae_hist

```